Resilient Communication Networks

Designing Resilient Communication Networks

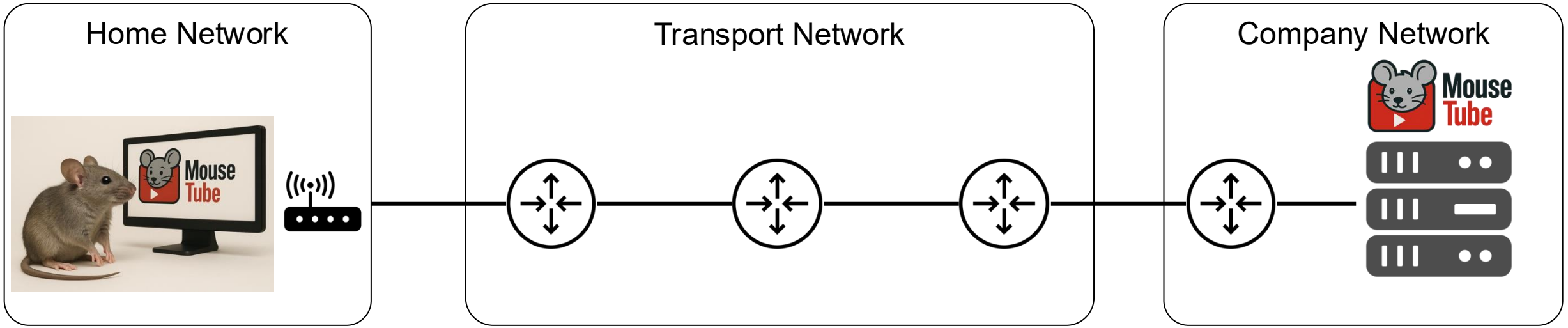
Redundancy Mechanisms – Transport Layer



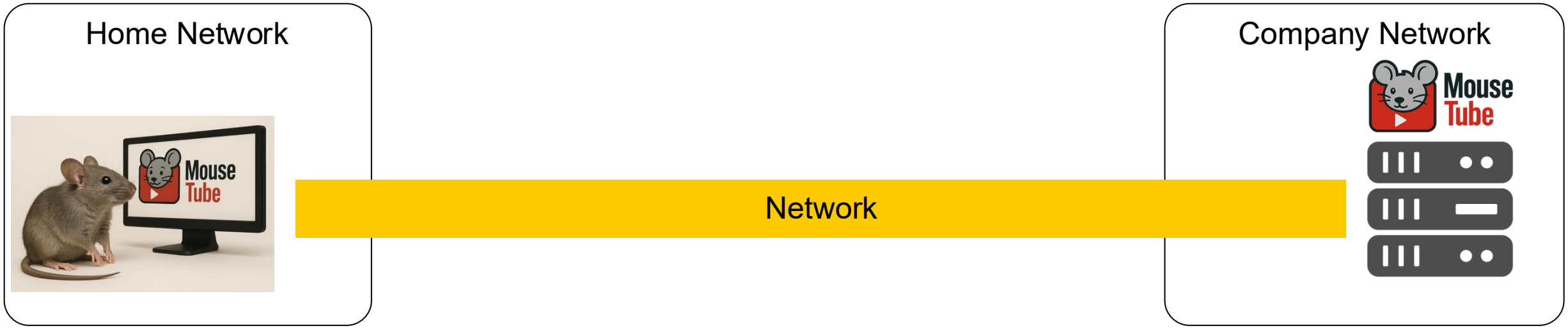
KOM – Communication Networks Lab
Technical University of Darmstadt
Prof. Dr. Björn Scheuermann (Director)
Dept. of Electrical Engineering and Information Technology
Dept. of Computer Science (adjunct Professor)
www.KOM.tu-darmstadt.de

Disclaimer: some of these slides have been derived from Günter Schäfer's lecture „Protection of Communication Infrastructures", Mathias Fischers Lecture „Resilient Networks", Matthias Hollick's Lecture „ Ringvorlesung KRITIS", and Ralf Steinmetz's Lecture "Communication Networks II"

What components would need to be redundant?



What components would need to be redundant?



Multi-Homing

TECHNISCHE UNIVERSITÄT DARMSTADT

Idea for network access: One device has more than one possibilities to access a network

Often improves reliability and performance

- If one of the networks is congested or failed, a failover to the other network can be performed
- It must be ensured that no single-point-of-failure exists beyond the connection itself
  - DNS-Servers for name resolution
  - Database access or other services
  - Dedicated routers / switches

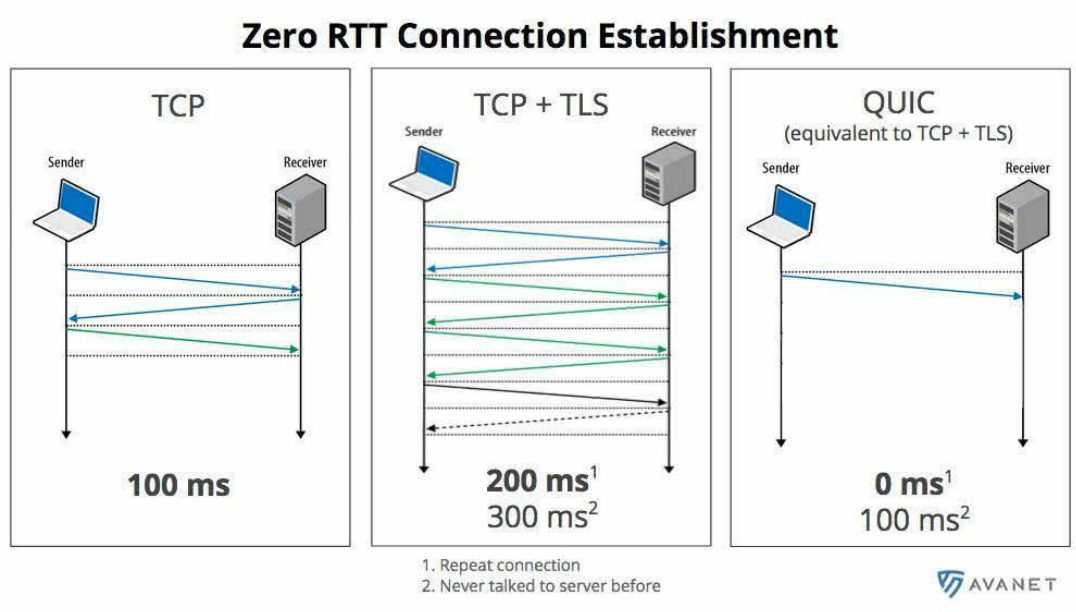
For example

- Smartphones use cellular and Wifi simultaneously
- Many desktop machines can use Wifi and wired connections simultaneously

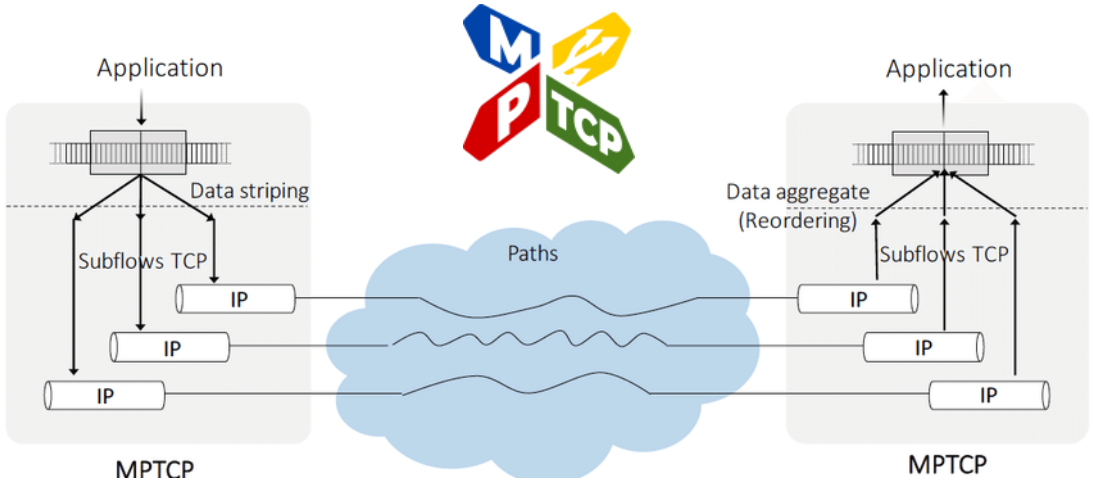
How can multihoming be utilized for fast failover?

Handling IP-address changes after failovers

QUIC

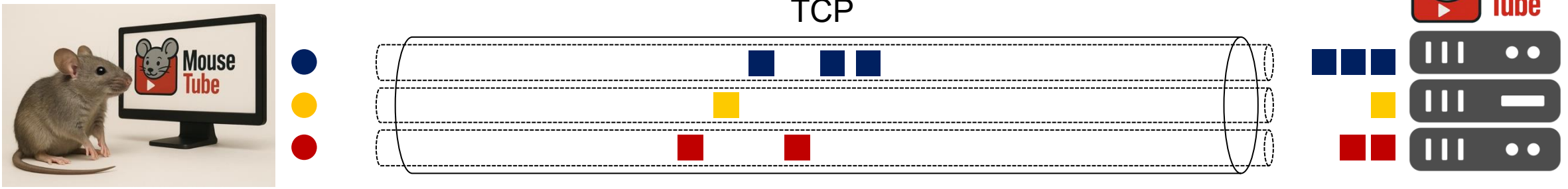


Multipath TCP (MPTCP)



https://www.researchgate.net/publication/316450813/figure/fig1/AS:486885853274112@1493094041081/Multipath-TCP-Connection-Overview.png
https://www.avanet.com/assets/sophos-firewall-tcp-tcp-tls-quic.jpg

How does communication work with normal HTTP/2 via TCP?



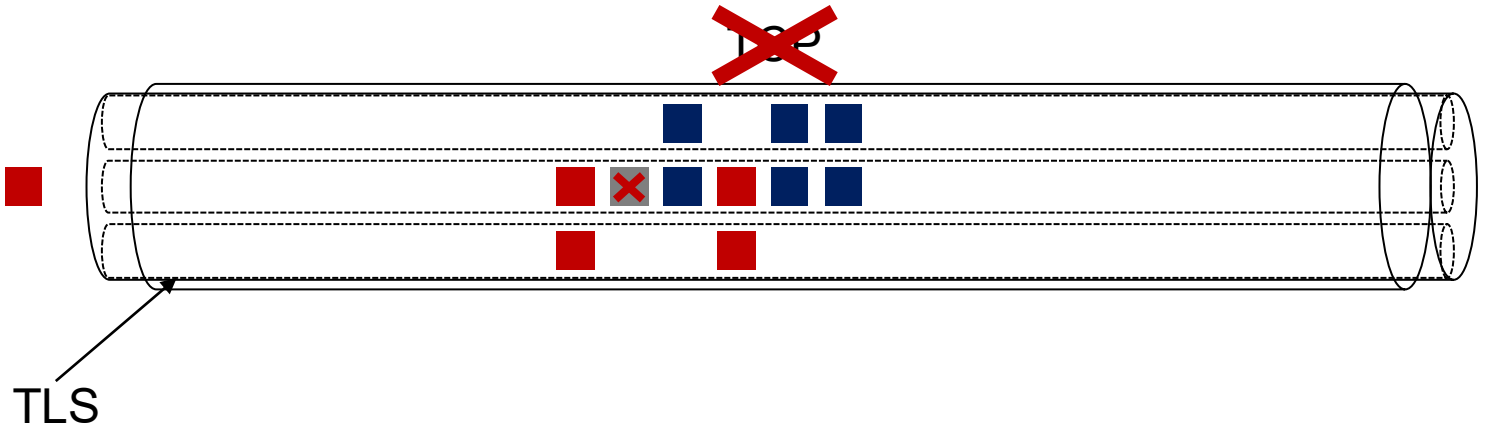
The client requests resources (e.g., HTML, CSS, and a video)

1. The client creates a TCP connection to the server
2. The client sends all requests via that TCP connection (but logically separated via channels)
3. The server replies to all requests in any order

Why did anyone bother to invent QUIC?

Challenges of normal HTTP/2 via TCP







Network change

- If the client IP changes, the TCP connection breaks
- After that, it needs to be reestablished

TLS-Handshake after connection setup

- Multiple handshakes are required for communication
- That increases latency of the first packet

Head-of-Line blocking

- TCP guarantees the order of packets (even if it may not be necessary)
- If a packet gets lost, all streams are paused

Where do all these issues come from?

TECHNISCHE UNIVERSITÄT DARMSTADT

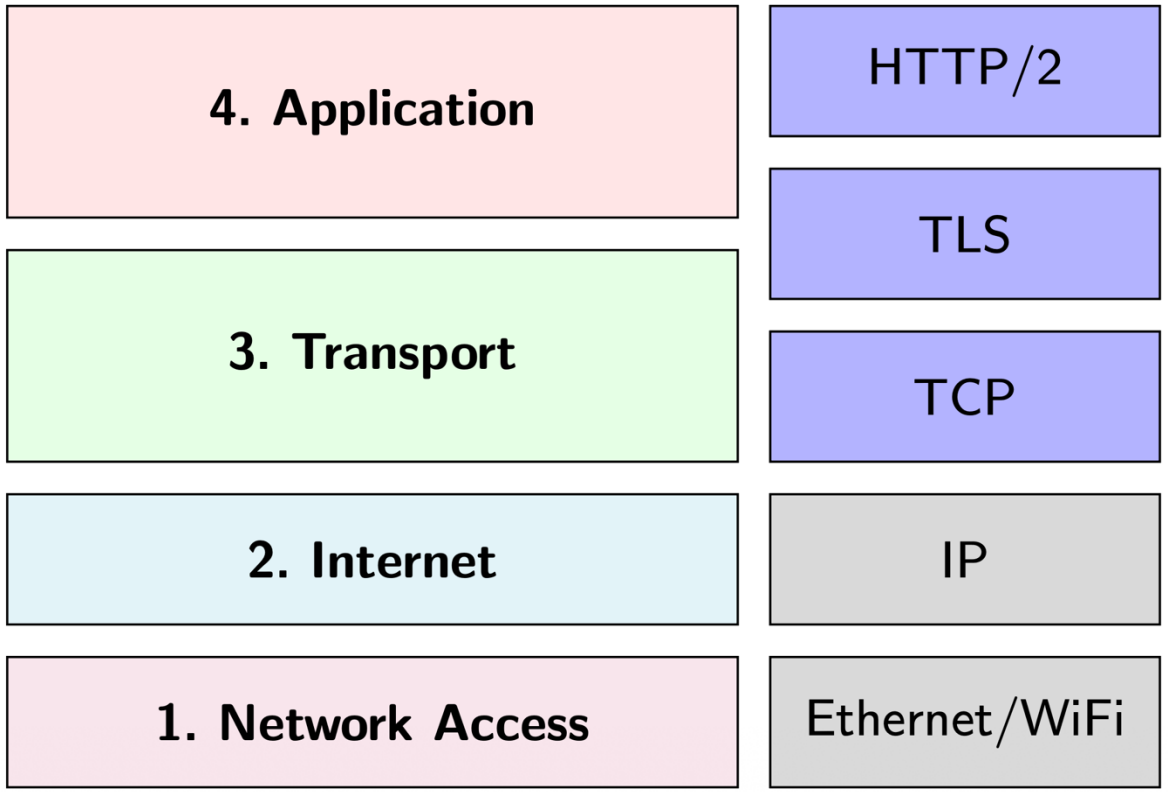
TCP is widely used

- In different applications
- Fundamental changes are hard to make

The transport layer (in this case TCP) does not receive information of the requirements of upper/lower layers

- Additional handshake for TLS
- Applications requirements of, e.g., HTTPS

Modern protocols (like QUIC) try to consider these requirements



QUIC in the protocol stack

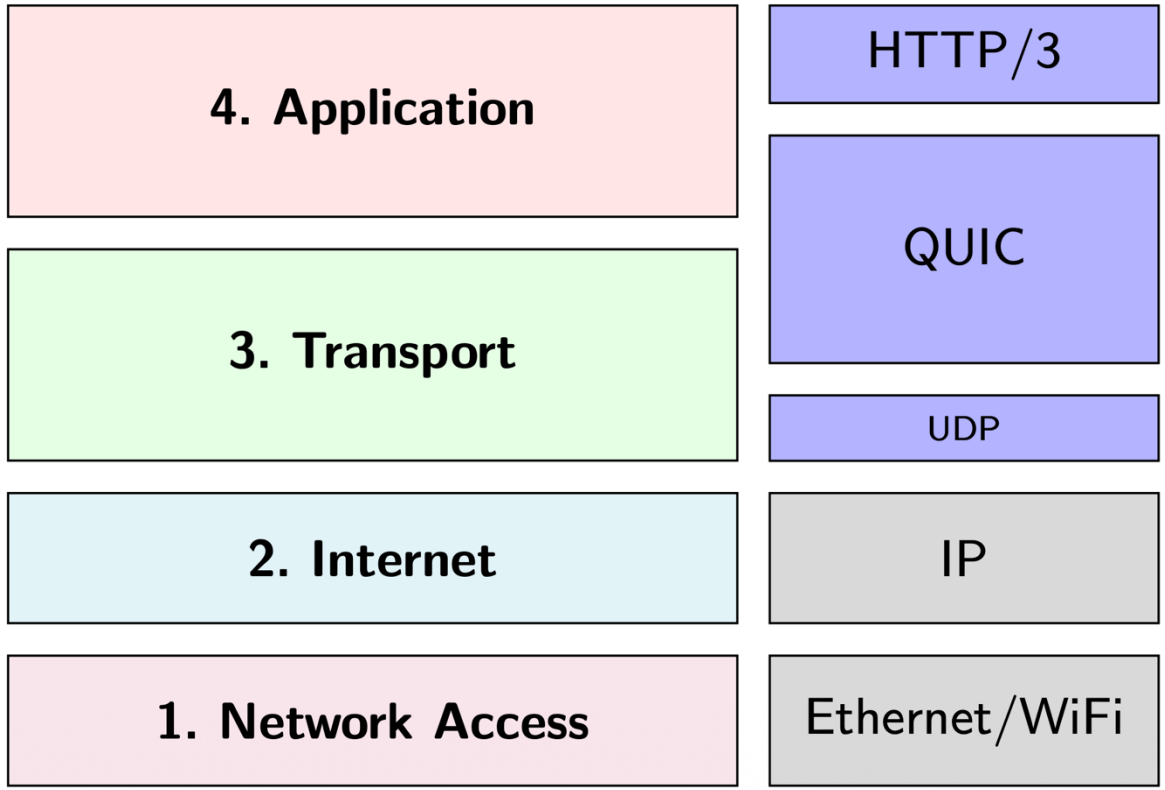
TECHNISCHE UNIVERSITÄT DARMSTADT

QUIC combines functionality of TCP, TLS and HTTP/2

- Flow and congestion control like TCP
- Encryption like TLS
- Multiplexing like HTTP/2

It builds upon UDP and complements it with TCP functionality

Why is QUIC based on UDP and not on IP?



Challenges of new transport layer protocols

TECHNISCHE UNIVERSITÄT DARMSTADT

In the optimal case, QUIC would be built on top of IP. However:

- Existing middle boxes might not know QUIC and block it
- Such incompatabilities can severly harm the deployment of new protocols

For example, nftables (a standard linux tool) filters on network and transport layer

IP filter	TCP filter	UDP filter
Source/destination IP	Source/destination port	Source/destination port
Protocol type	TCP flags (SYN, ACK)	- (no flags)
Interface	Connection tracking	Connection tracking

As firewalls allow protocols explicitly, new protocols are often not supported for an extended duration

Overhead caused by using UDP instead of IP

TECHNISCHE UNIVERSITÄT DARMSTADT

QUIC builds upon UDP for that reason

- Compatibility with firewalls and middleware
- Receiving and sending of packets in the operating system

UDP has almost no overhead compared to IP packets

- UDP header is only 8 bytes
- The respective header fields would anyways be required for QUIC

Source port (16 bit)	Destination port (16 bit)
Length (16 bit)	Checksum (16 bit)
Data (variable length)	

QUIC Header

Not relevant for the exam

Long Header (before handshake)

- Information for header identification (8 bit)
- Version (32 bit)
- Destination connection ID (0-160 bit)
- Source connection ID (0-160 bit)
- Further information depending on the header type

Short Header (after handshake)

- Information for header identification (8 bit)
- Destination connection ID (160 bit)
- Packet number (8-32 bit)



QUIC Header

Not relevant for the exam

Long Header

Header Form	Fixed Bit	Long Packet Type	Type Speicific bits	Version ID	DCID Len	DCID	SCID Len	SCID
1 bit	1 bit	2 bits	4 bits	32 bits	8 bits	0-160 bits	8 bits	0-160 bits

Short Header

Header Form	Fixed Bit	Spin bits	Reserved	Key Phase	P	DCID	Packet Number	Protected payload
1 bit	1 bit	1 bit	2bits	1 bit	2 bits	160 bits	P+8 bits	

QUIC as transport layer protocol on application layer

TECHNISCHE UNIVERSITÄT DARMSTADT

QUIC implementations are executed in user space, which enables easier adaptation without modifying the kernel

- Development of libraries for QUIC is much simpler than kernel modifications
- Lower effort for the installation of new changes
- Hoher requirements for interoperability between libraries

To use QUIC, you need to use a suitable library

Implementation	Language	Organisation
quiche	Rust	Cloudflare
msquic	C	Microsoft
aioquic	Python	Open Source (Jeremy Lainé)
quic-go	Go	Google
Chromium	C++	Google (Chromium project)

Usage of QUIC

TECHNISCHE UNIVERSITÄT DARMSTADT

Google QUIC (gQUIC) was initially developed by Google

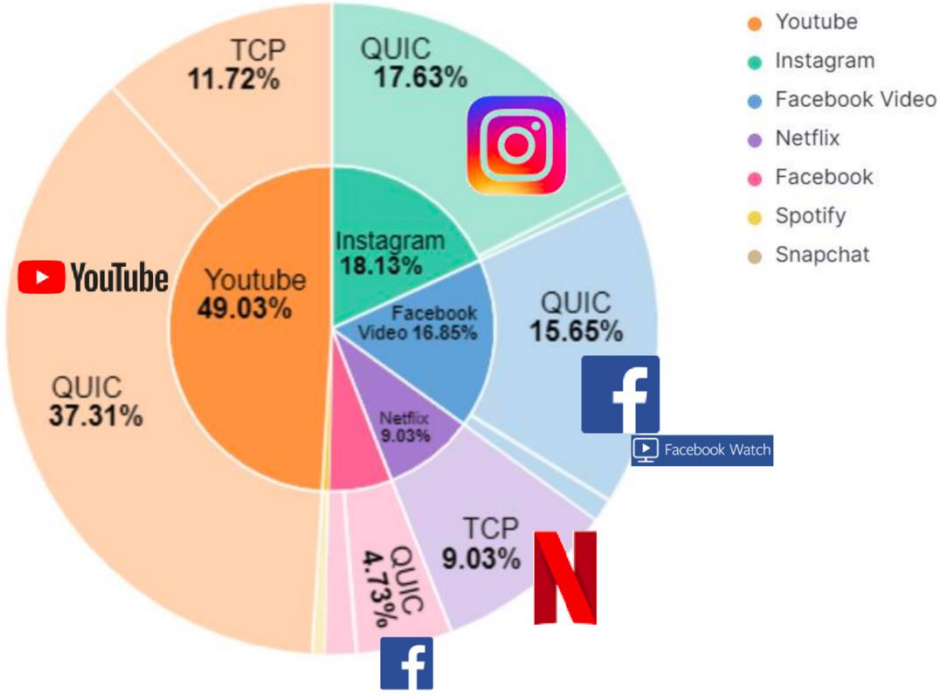
- Used for communication between Google products
- Based on custom security and flow control mechanisms

In 2016, Google proposed gQUIC to the IETF

- QUIC (without the “g”) is then standardized with significant changes, e.g., the usage of TLS 1.3

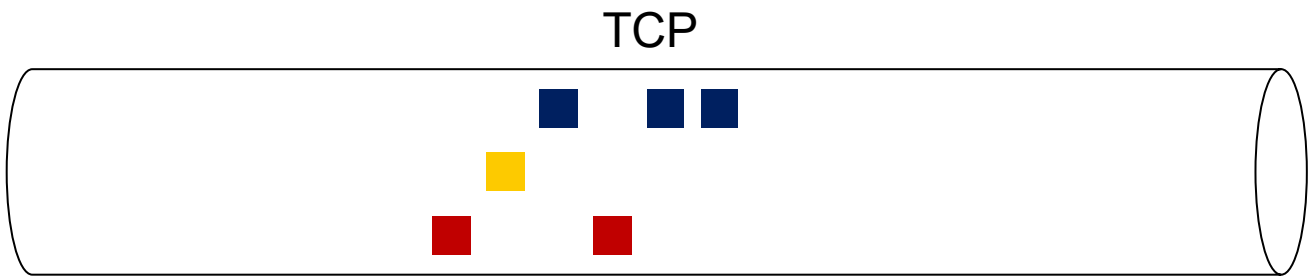
Many large operators and cloud providers support QUIC

- QUIC is the standard protocol of HTTP/3
- The majority of traffic of the large providers utilizes QUIC



Approaches of QUIC to the challenges of HTTP/2







Network change

TLS-Handshake

Head-of-Line blocking

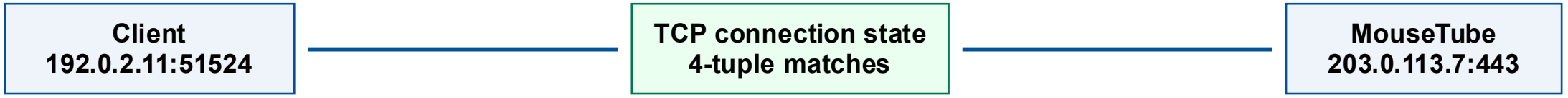
Network change: what breaks in TCP?

TECHNISCHE UNIVERSITÄT DARMSTADT

TCP identifies a connection by source IP, destination IP, source port, and destination port

- When WiFi fails and the client switches to cellular, that tuple changes
- The server therefore cannot attach the new packets to the old connection: the application must reconnect

Before failover: WiFi path is active



After failover: cellular path has a new client IP

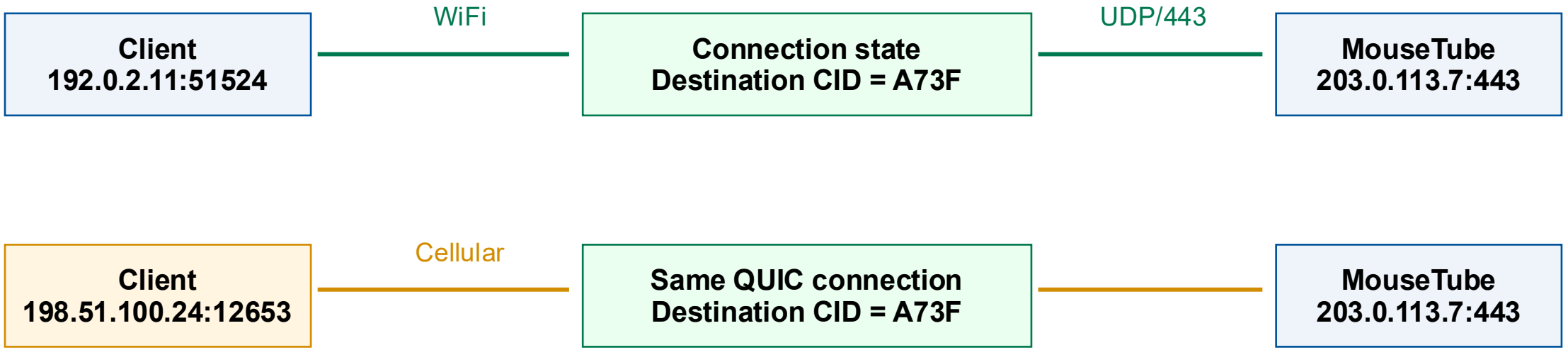


QUIC connection migration: same connection, new path

TECHNISCHE UNIVERSITÄT DARMSTADT

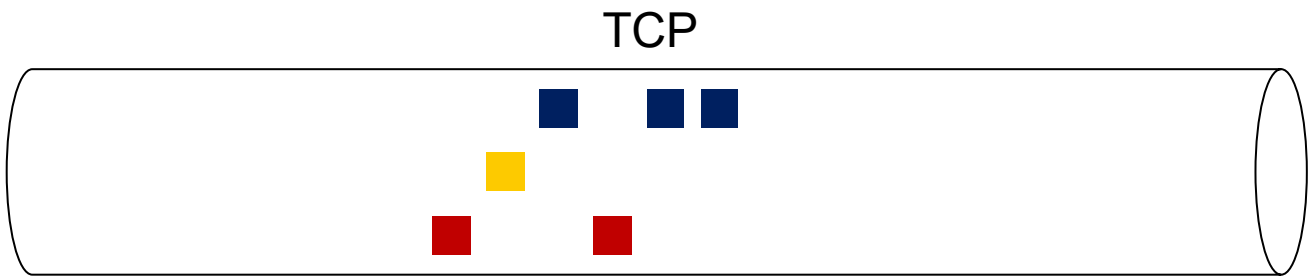
QUIC packets carry a Connection ID that is independent of IP addresses and UDP

- After the network changes, the server uses the Connection ID to find the existing connection state
- The client can keep the same encrypted streams instead of performing a full reconnect



Approaches of QUIC to the challenges of HTTP/2







Network change

TLS-Handshake

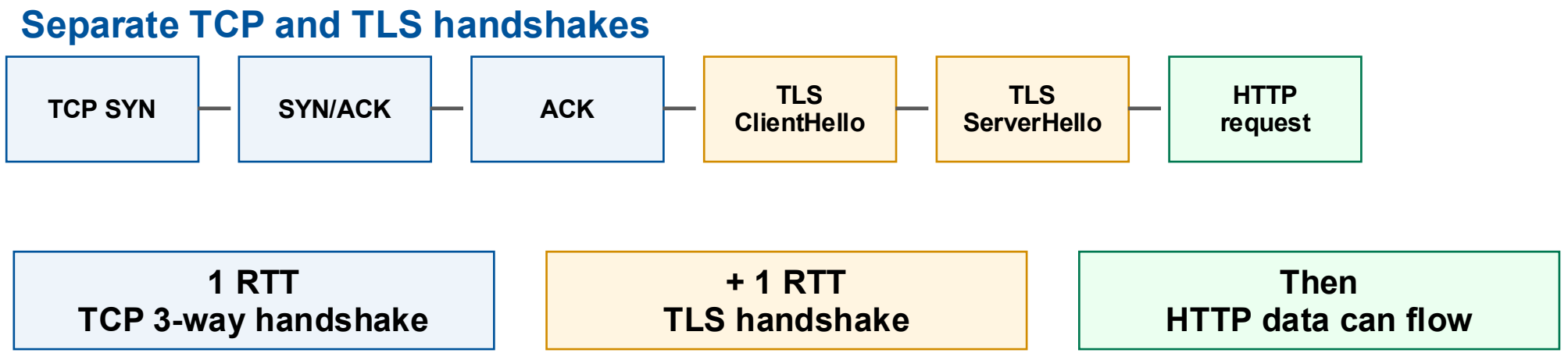
Head-of-Line blocking

TLS over TCP: why the first request is delayed

TECHNISCHE UNIVERSITÄT DARMSTADT

With HTTPS over TCP, the transport connection and the cryptographic session are established one after the other

- The HTTP request must wait until TCP is established and TLS has negotiated keys and authenticated the server
- This increases latency especially for short web transfers and after reconnects



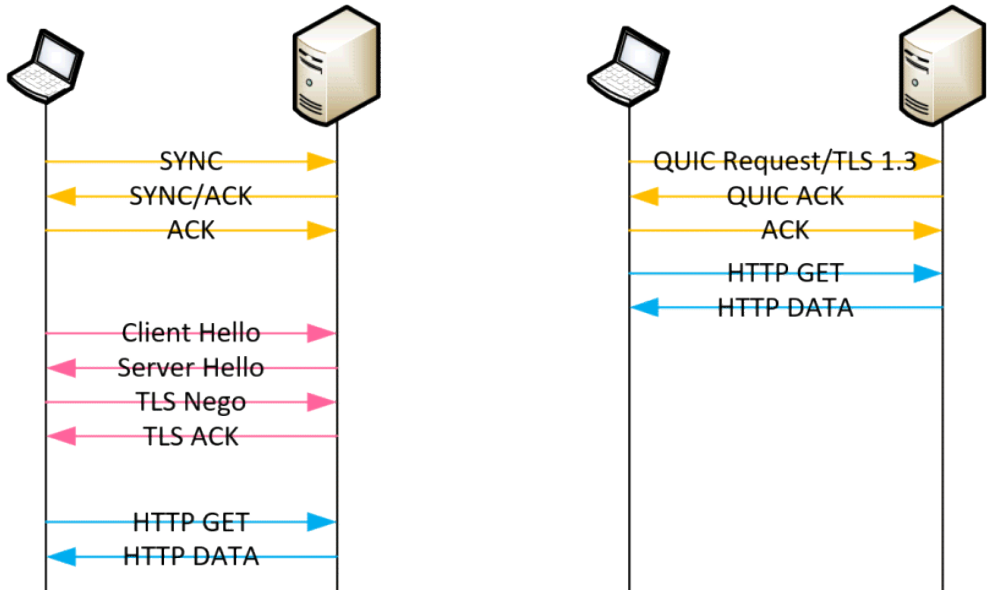
QUIC: TLS 1.3 is part of the transport handshake

TECHNISCHE UNIVERSITÄT DARMSTADT

QUIC establishes security and transport connections in parallel

- TLS 1.3 messages inside QUIC handshake packets
- The handshake also negotiates QUIC-specific information such as transport parameters and connection IDs

Result: QUIC can authenticate the server, negotiate encryption keys, and establish transport state together.



QUIC resumption and failover: fewer repeated handshakes

TECHNISCHE UNIVERSITÄT DARMSTADT

During a network change, QUIC keeps the existing cryptographic context and stream state, so the transfer can continue on the new path

- For future connections to the same server, QUIC can resume the connection and send selected early data immediately
- 0-RTT is not suitable for every request because early data can be replayed; safe/idempotent requests are allowed

Two different latency savings

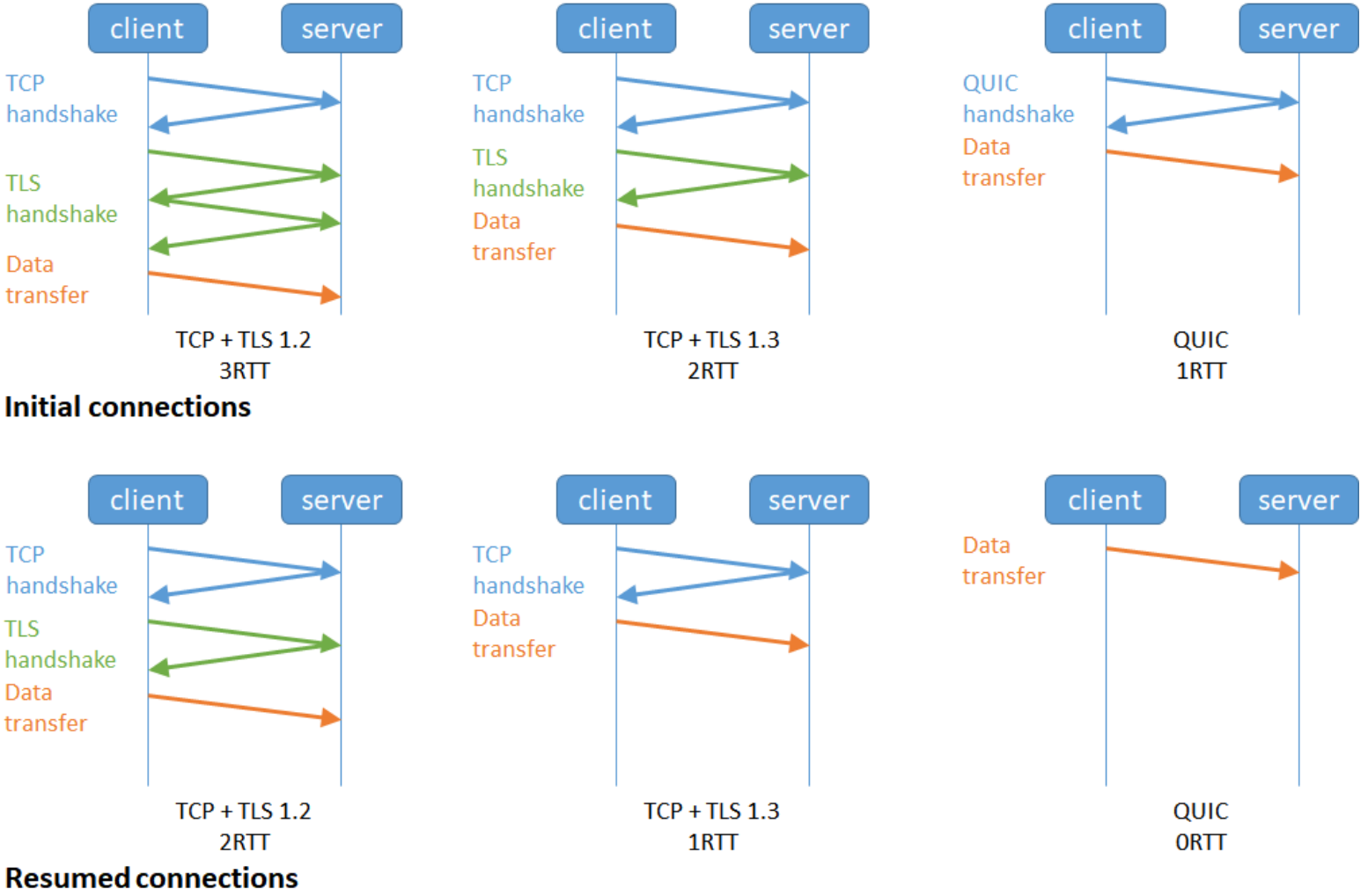
Connection migration
Active connection survives a network change
=> no new TCP/TLS setup

same connection ID

0-RTT resumption
A later new connection can send early data using stored resumption state

ClientHello + early HTTP data

TCP vs. UDP – Connection establishment



https://www.starwindsoftware.com/blog/wp-content/uploads/2021/06/word-image-31.png

0-RTT resumed connections

TECHNISCHE UNIVERSITÄT DARMSTADT

0-RTT connection establishment cannot exchange any keys before data transmission

- QUIC is an encrypted protocol

What keys do we use?

Idea: Use the already exchanged keys from previous sessions

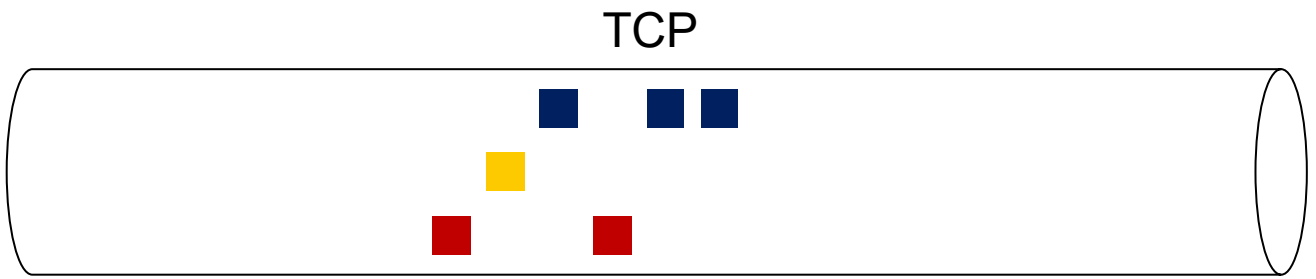
If 0-RTT is supported by the server, it provides a NewSessionTicket and sets the TLS early_data field to 0xffffffff

- This NewSessionTicket essentially contains information about the connection parameters
- Is stored on the client alongside other information necessary for connection resumption
- The NewSessionTicket is usually valid for 7 days and can be used during that time

Can we safely use 0-RTT connection resumption all the time?

Approaches of QUIC to the challenges of HTTP/2







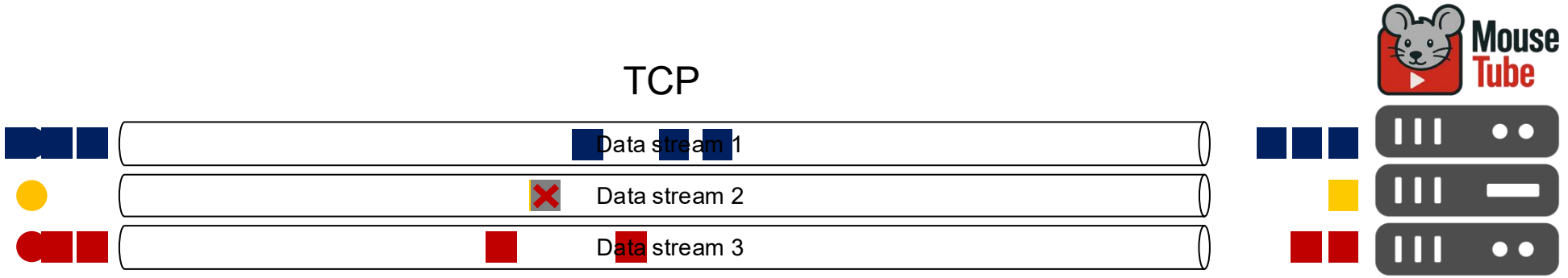
Network change

TLS-Handshake

Head-of-Line blocking

Stream-based Multiplexing in QUIC





QUIC create parallel, independent data streams

The client requests resources (e.g., HTML, CSS, and a video)

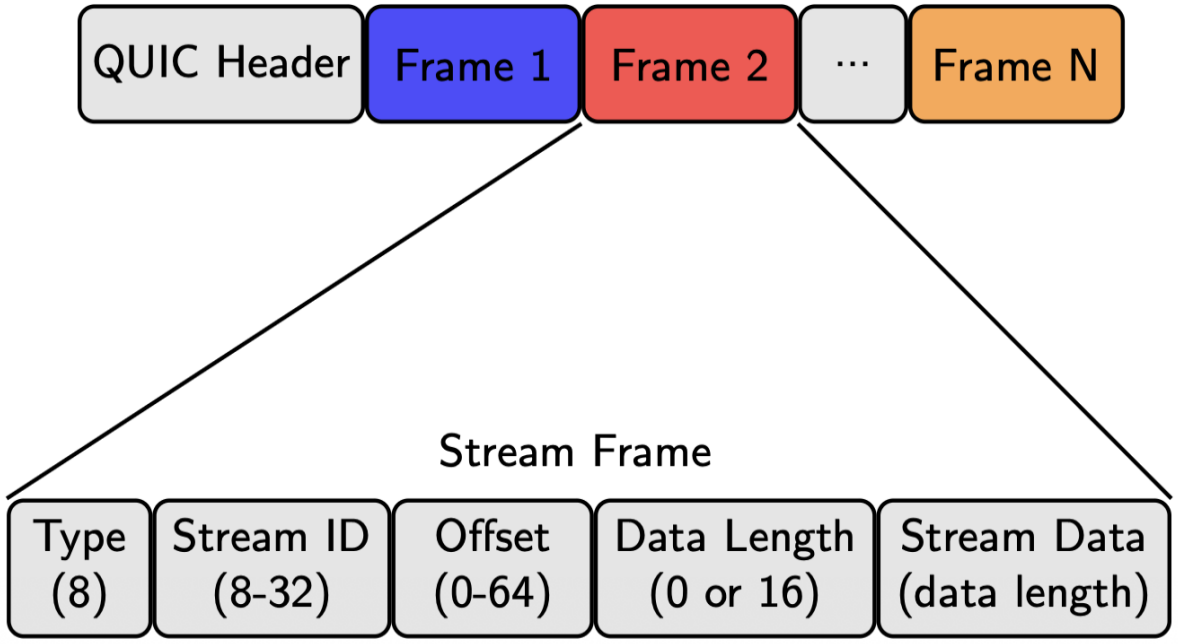
- Each data stream has its own flow control and loss handling
- Losses or delays of the other streams have no influence

QUIC Packet Structure

TECHNISCHE UNIVERSITÄT DARMSTADT

A QUIC packet consists of the QUIC header and one or more stream frames

- Multiple frames (of different streams) can be enveloped into a single QUIC packet

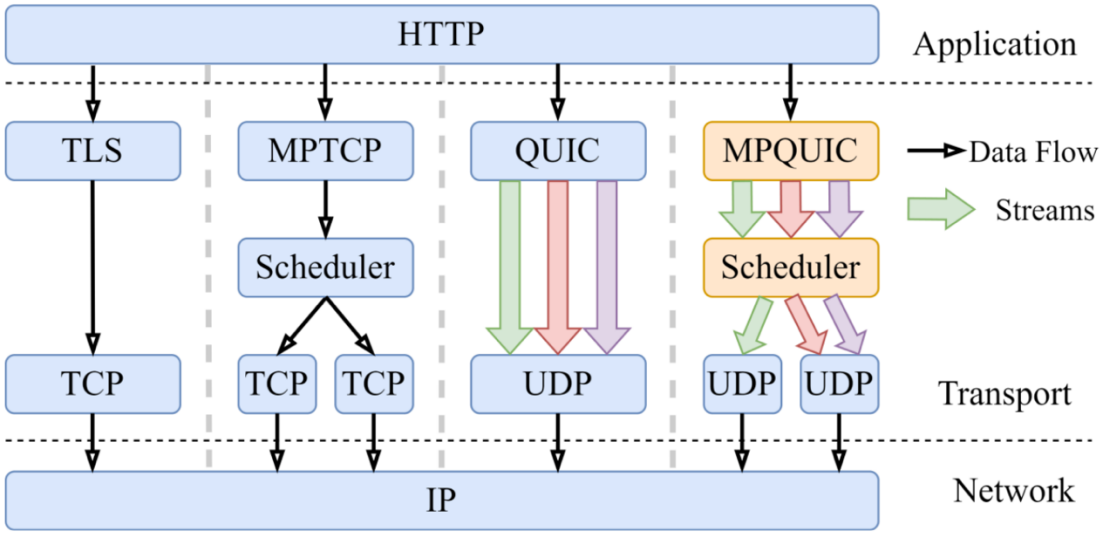


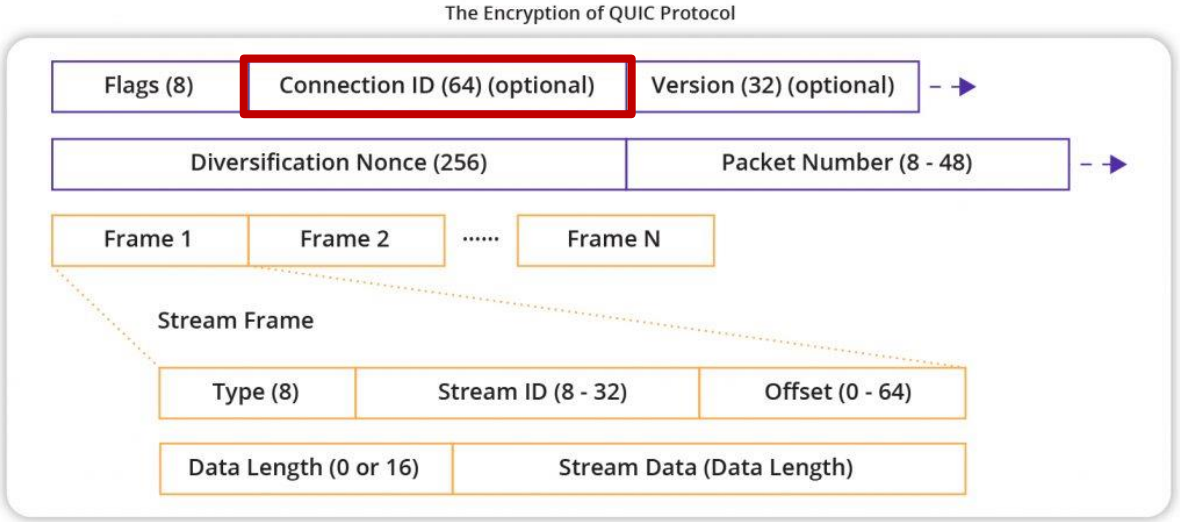
Multipath QUIC

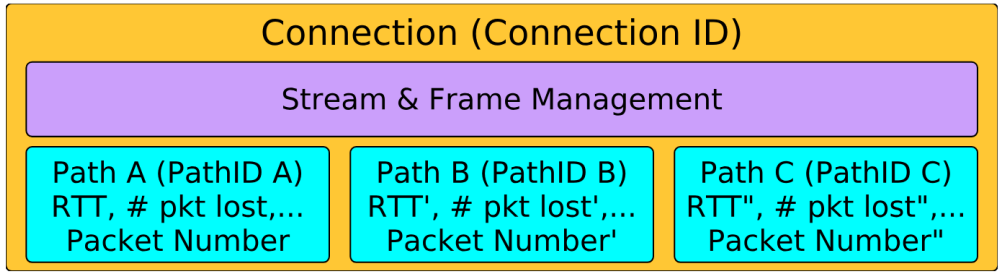
TECHNISCHE UNIVERSITÄT DARMSTADT

Similar to MPTCP, QUIC also has a multipath extension (MPQUIC)

- MPQUIC follows the design logic of MPTCP and inherits the essential feature in QUIC
- connection establishment, stream multiplexing, and frame structures







De Coninck, Quentin, and Olivier Bonaventure. "Multipath quic: Design and evaluation." In *Proceedings of the 13th international conference on emerging networking experiments and technologies*, pp. 160-166. 2017.

https://www.nsnam.org/workshops/wns3-2023/thursday/11-shu-slides-wns3-2023.pdf

Path Identification

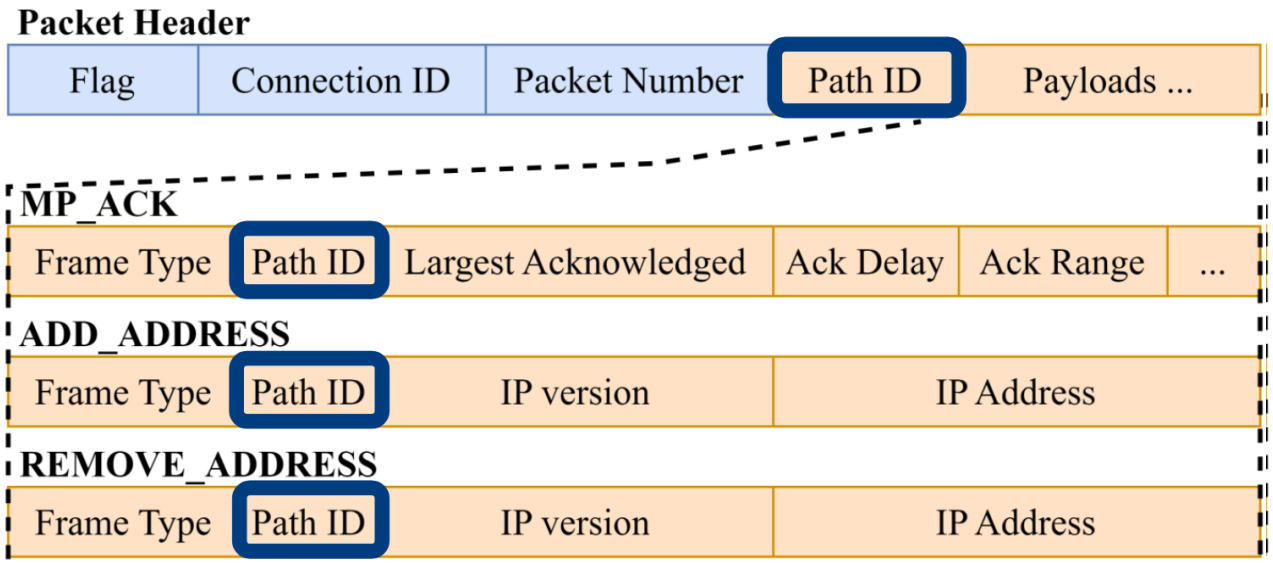
TECHNISCHE UNIVERSITÄT DARMSTADT

Host need to manage multiple paths simultaneously

- Similarly as in MPTCP, middleboxes might change attributes of the connection
- For example, middleboxes might drop packets with old sequence numbers

MPQUIC adds the path ID to the public header

- Middleboxes can consider it when inspecting packets
- Allows to keep connection information even after an IP address change



Quentin De Coninck and Olivier Bonaventure. 2017. Multipath QUIC: Design and Evaluation. In Proceedings of the 13th International Conference on emerging Networking EXperiments and Technologies (CoNEXT '17). Association for Computing Machinery, New York, NY, USA, 160–166
https://www.nsnam.org/workshops/wns3-2023/thursday/11-shu-slides-wns3-2023.pdf

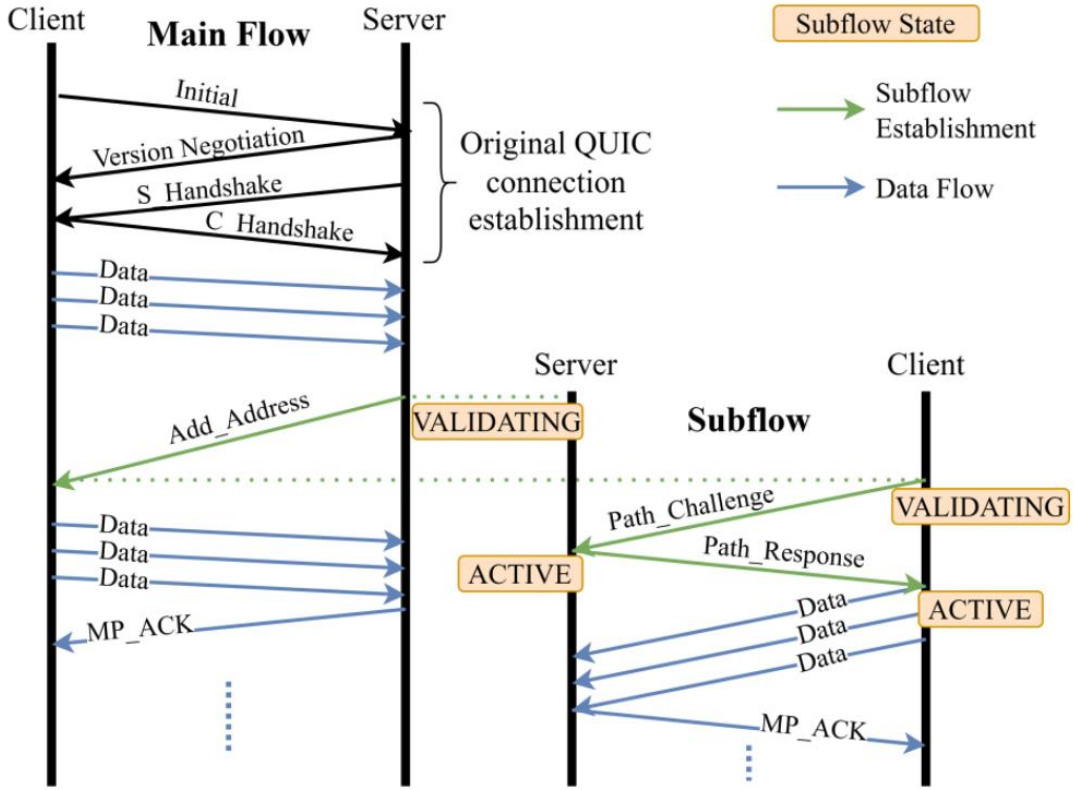
Path Establishment

QUIC is all about security

How do you prevent that someone just requests a path without permissions?

QUIC utilizes path challenges

- Verifying that the device receiving the packets along this path is actually the one with established security handshake



Quentin De Coninck and Olivier Bonaventure. 2017. Multipath QUIC: Design and Evaluation. In Proceedings of the 13th International Conference on emerging Networking EXperiments and Technologies (CoNEXT '17). Association for Computing Machinery, New York, NY, USA, 160–166
https://www.nsnam.org/workshops/wns3-2023/thursday/11-shu-slides-wns3-2023.pdf

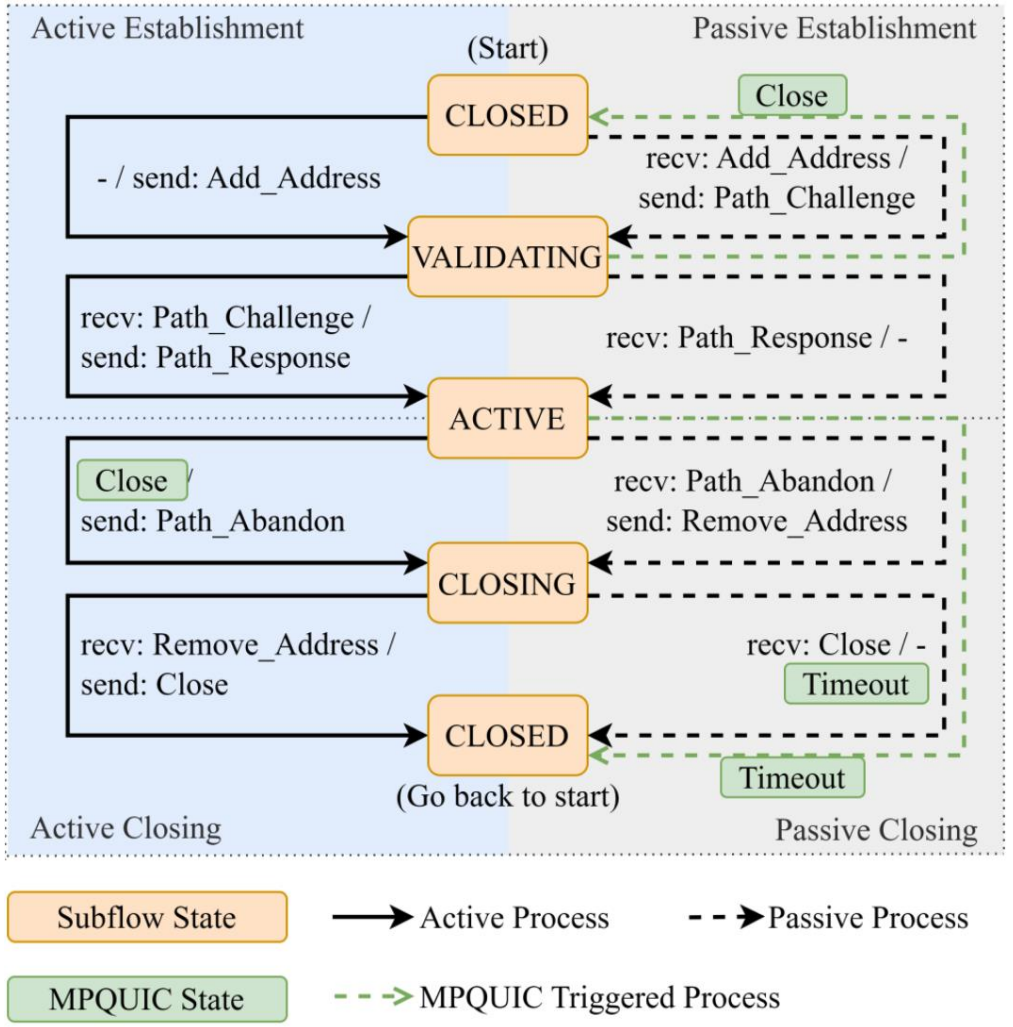
Path Establishment

QUIC is all about security

How do you prevent that someone just requests a path without permissions?

QUIC utilizes path challenges

- Verifying that the device receiving the packets along this path is actually the one with established security handshake



Quentin De Coninck and Olivier Bonaventure. 2017. Multipath QUIC: Design and Evaluation. In Proceedings of the 13th International Conference on emerging Networking EXperiments and Technologies (CoNEXT '17). Association for Computing Machinery, New York, NY, USA, 160–166
https://www.nsnam.org/workshops/wns3-2023/thursday/11-shu-slides-wns3-2023.pdf

Summary for QUIC

TECHNISCHE UNIVERSITÄT DARMSTADT

Originally developed by Google, later standardized by IETF

- First support by YouTube and Chrome
- Nowadays wide support across browsers

QUIC is technically not a transport layer protocol

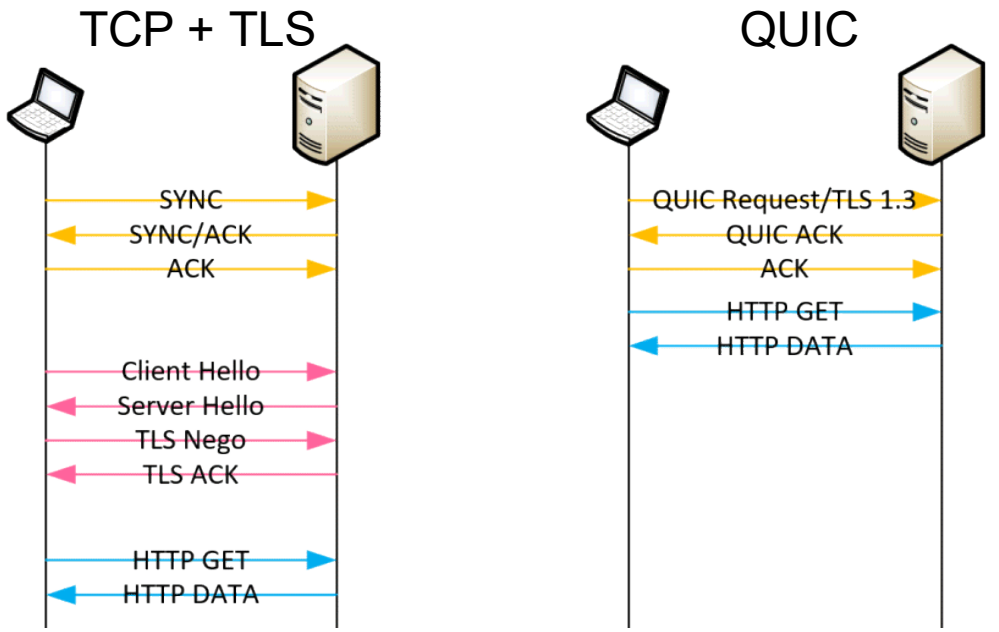
- Utilizes UDP on transport layer

QUIC only allows for encrypted communication

- Gaining efficiency by combining transport layer and crypto handshake

QUIC uses an explicit connection identifier instead of source and destination IP + port

- Allows for 0-RTT failover due to the availability of an active connection



https://www.msxfaq.de/netzwerk/quic/was_ist_quic04.gif

Multipath TCP

TECHNISCHE UNIVERSITÄT DARMSTADT

General idea: If more than one network interface are available, create a TCP connection on each interface

MPTCP is an evolution of TCP

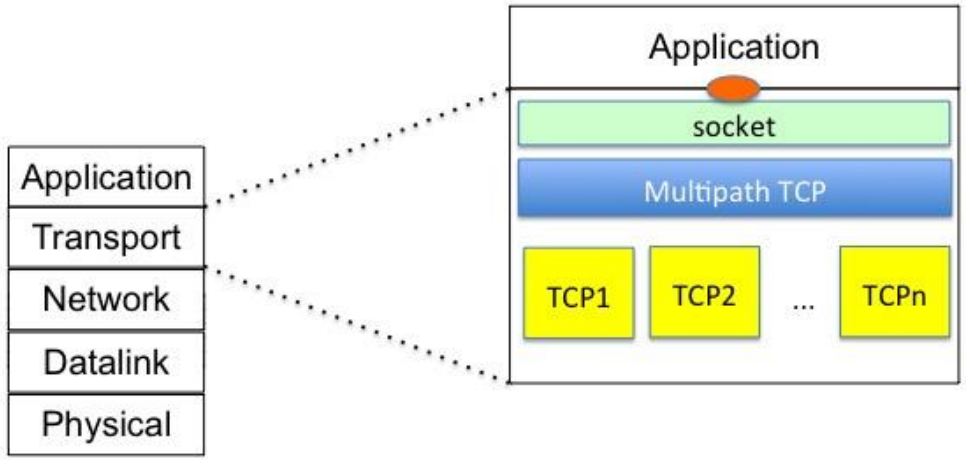
- Supports applications without any modifications
- Appears similar to the underlying network infrastructure

MPTCP requires additional coordination transmitted in TCP options with Option Kind 30

- May be influenced by middle-boxes
- If any modifications are done, MPTCP falls back to TCP

MPTCP manages multiple TCP connections

- Ensure ordered delivery
- Manage failover events



https://en.wikipedia.org/wiki/Multipath_TCP#/media/File:Multipath_TCP_architecture.jpg

TCP – Message Format

TECHNISCHE UNIVERSITÄT DARMSTADT

Reliable bidirectional in-order byte stream

- Connections can be established & torn down

Error control

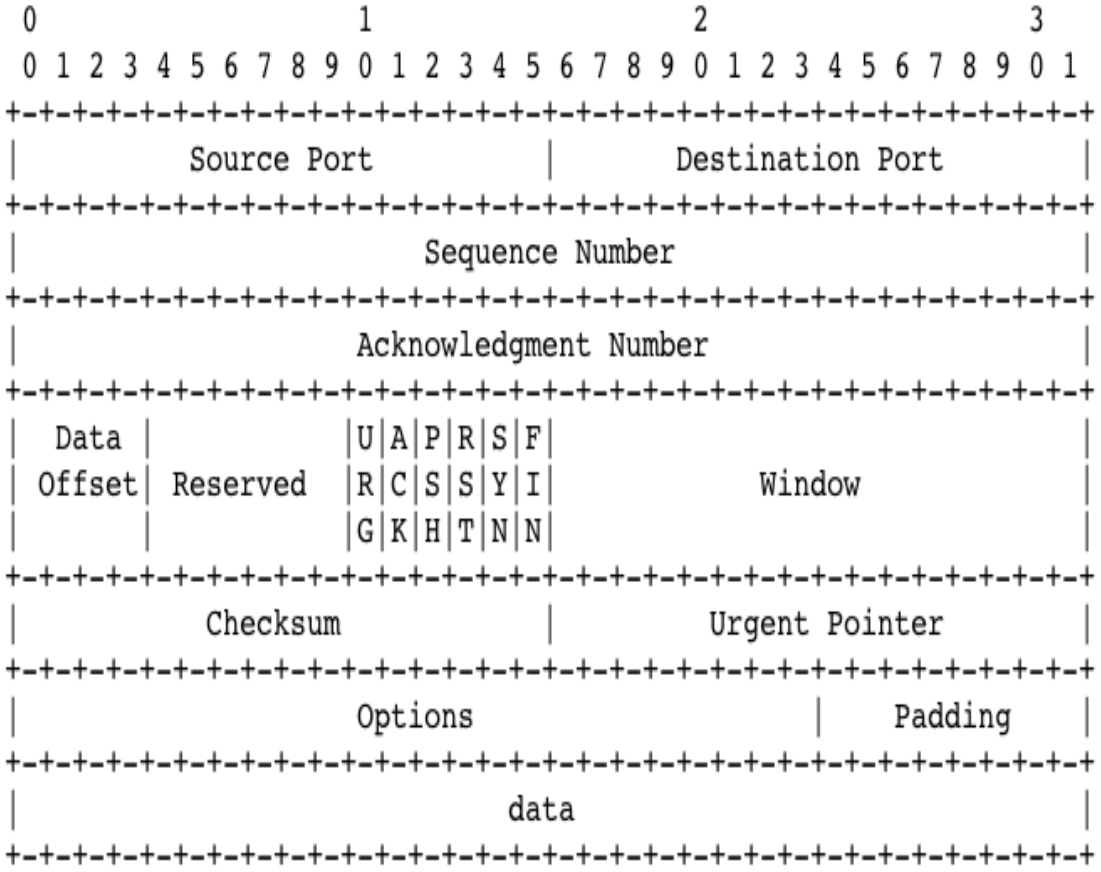
- users see correct, ordered byte sequences

End-to-end flow control

- avoid overwhelming the machines at either end

Congestion avoidance

- avoid creating traffic jams within network



TCP Header Format

Message Format

Not relevant for the exam

Variable length header

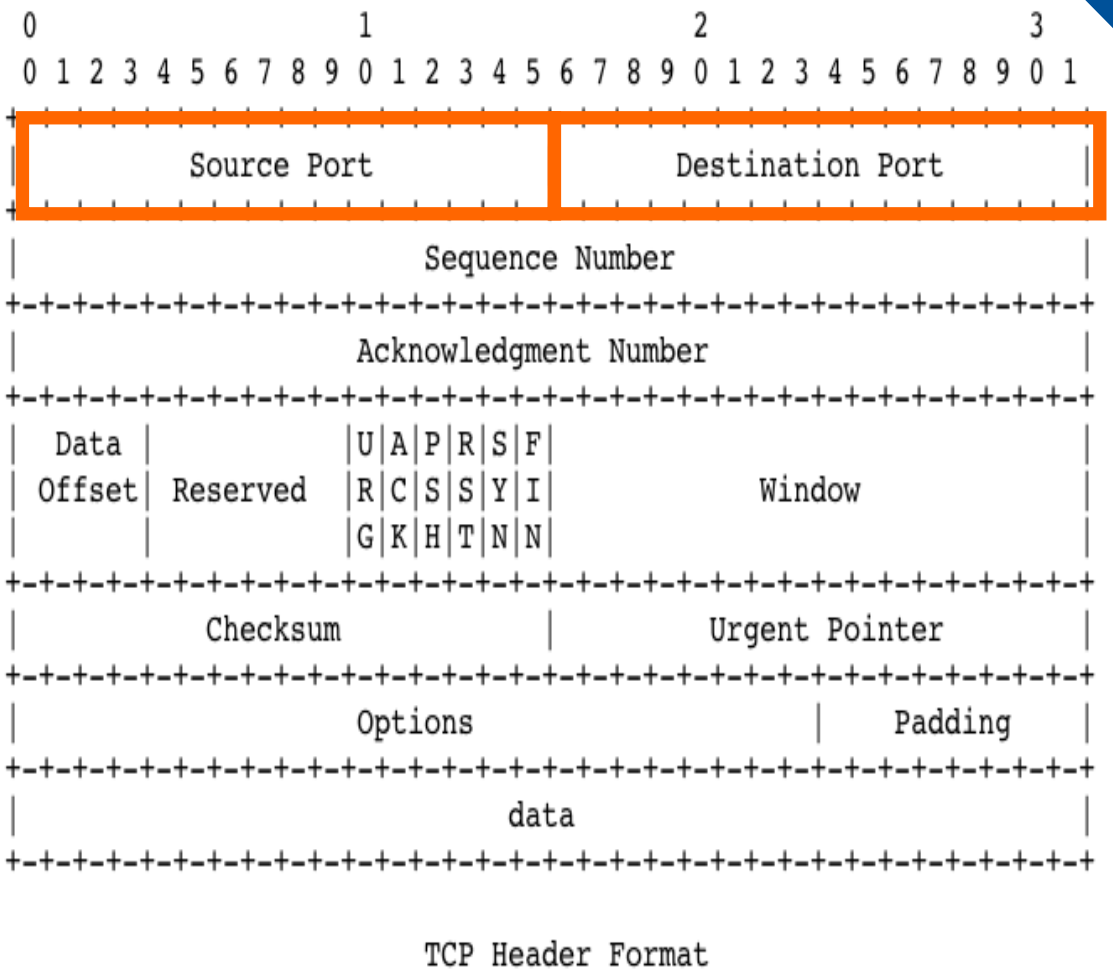
- Min. 20 byte
- Variable length options
- Multiple of 4 byte

Source port

- 16 bit identifier of sending application

Destination port

- 16 bit identifier of receiving application



TCP Header Format

Image source: http://www.rfc-editor.org/rfc/rfc793.txt

Not relevant for the exam

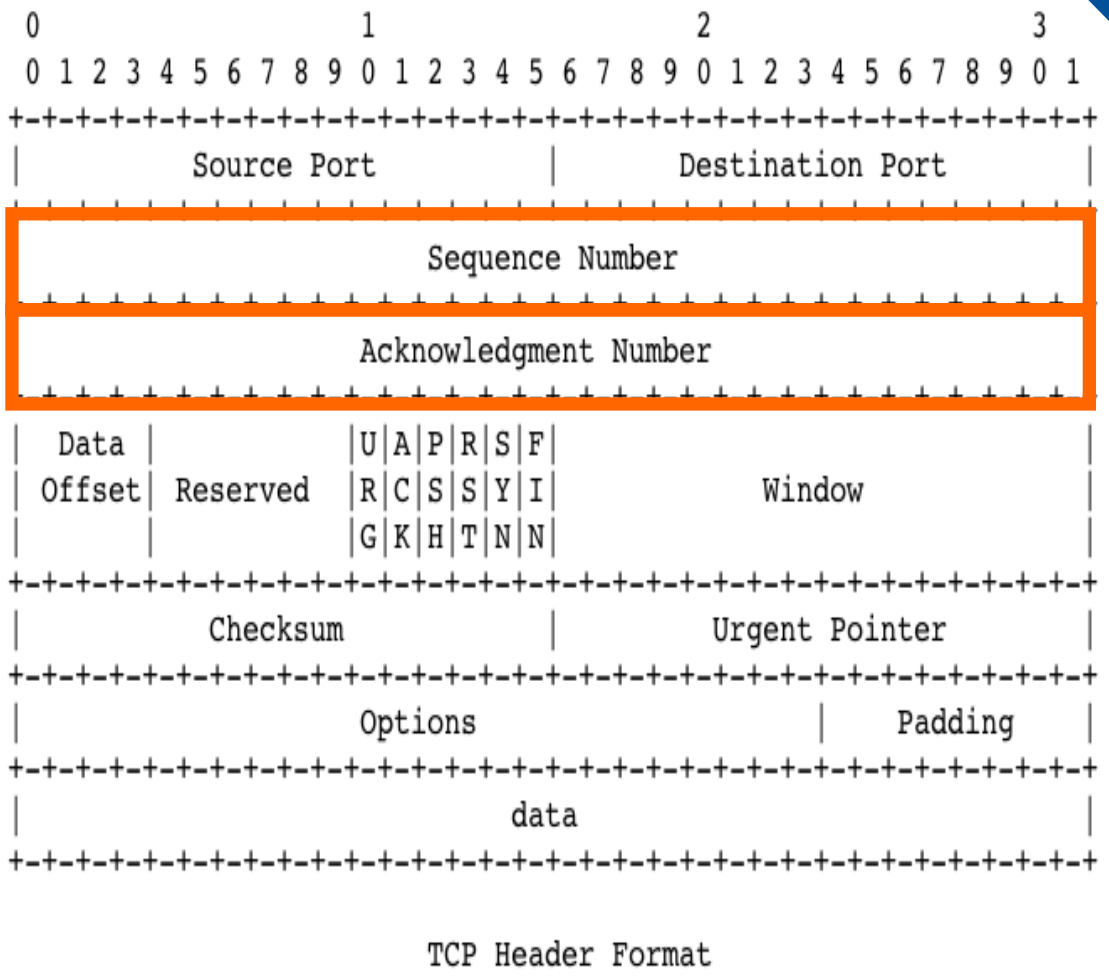
Message Format

Sequence number

- Connection setup: Initial sequence number negotiation
- Data transfer: Number of first byte in data field

Acknowledgment number

- Connection setup: Initial sequence number negotiation
- Data transfer: Next expected byte
  - Cumulative acknowledgment



TCP Header Format

Image source: http://www.rfc-editor.org/rfc/rfc793.txt

Message Format

Not relevant for the exam

Window

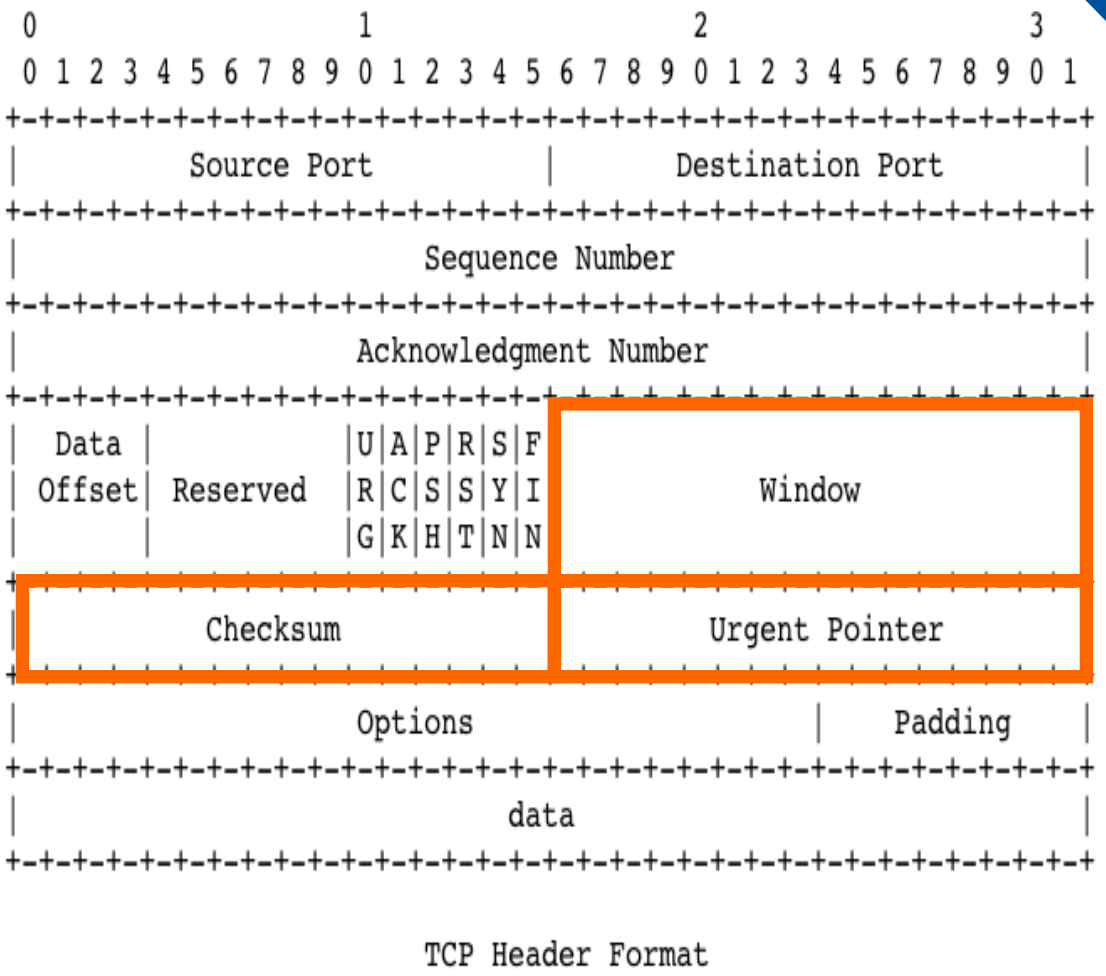
- Number of bytes sender of message can accept
- Indicates available buffer space of sender
- Used for flow control

Checksum

- Used for error detection
- Same recipe as UDP
  - But, mandatory

Urgent pointer

- Number of urgent bytes in data field
- Not used in practice



TCP Header Format

Image source: http://www.rfc-editor.org/rfc/rfc793.txt

Not relevant for the exam

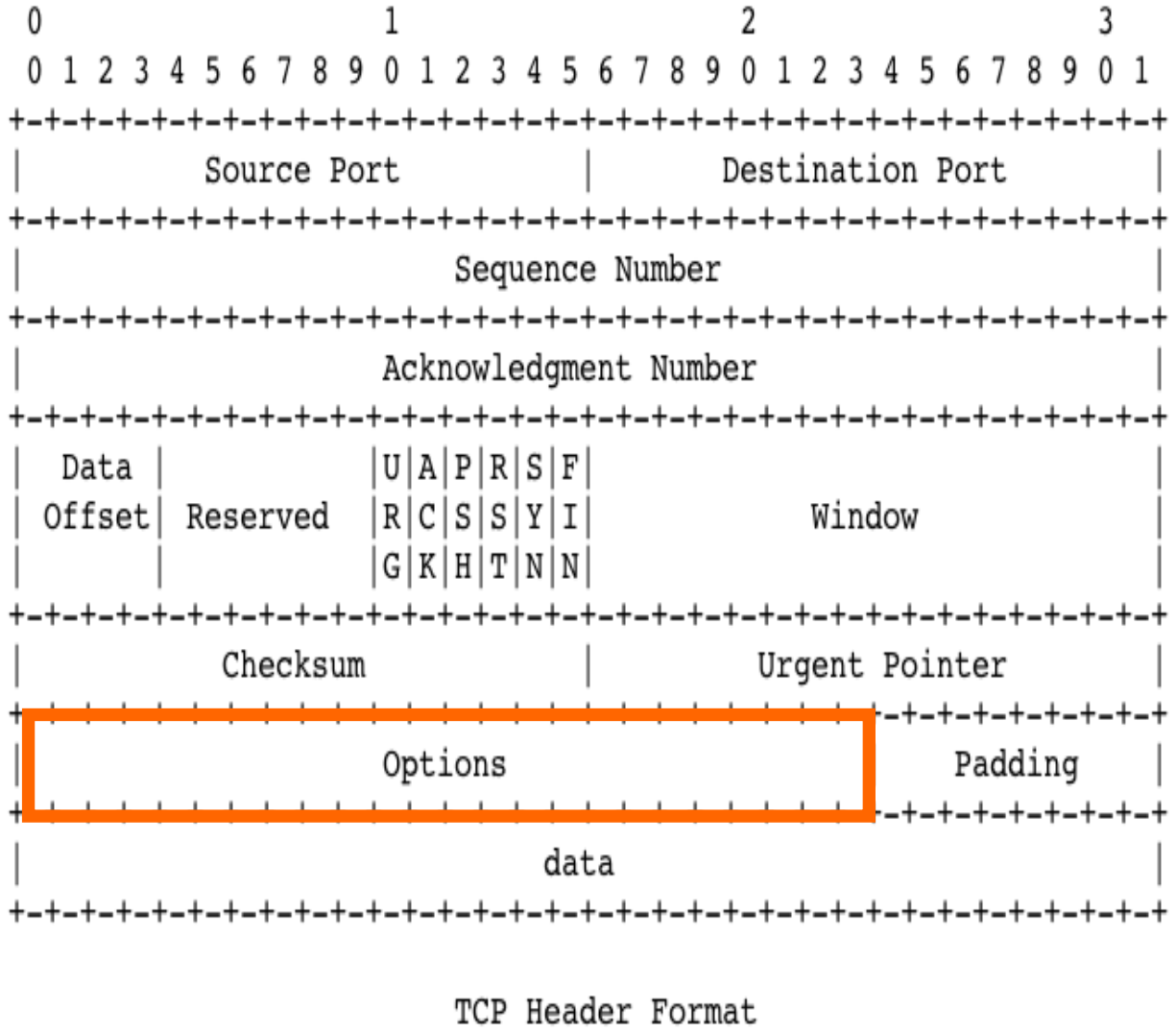
Message Format

Options

- Definition of protocol options
- Valid for connection

- E.g. mandatory , i.e. all implementations MUST support it
  - Definition of max. segment size
  - …. -> continued … at next slide

- E.g., really optional, depending also on the specific implementation
  - Selective acknowledgment
    - Useful in certain cases of packet loss
  - Window scaling
    - Original window size limited to 2^{16}= 2 EE16 = 65535 bytes
      - Scaling factor used to optimize TCP for high bandwidth connections



TCP Header Format

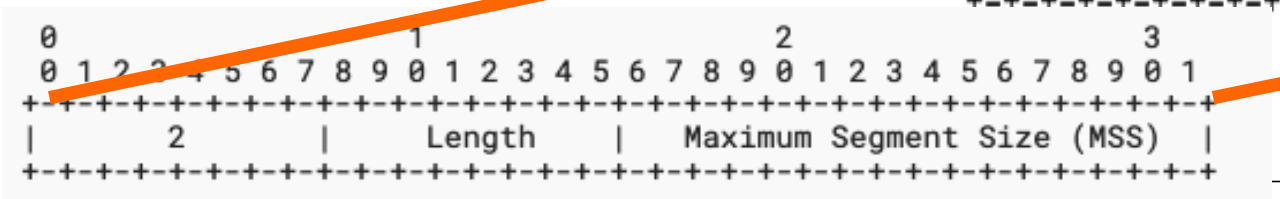
Image source: http://www.rfc-editor.org/rfc/rfc793.txt

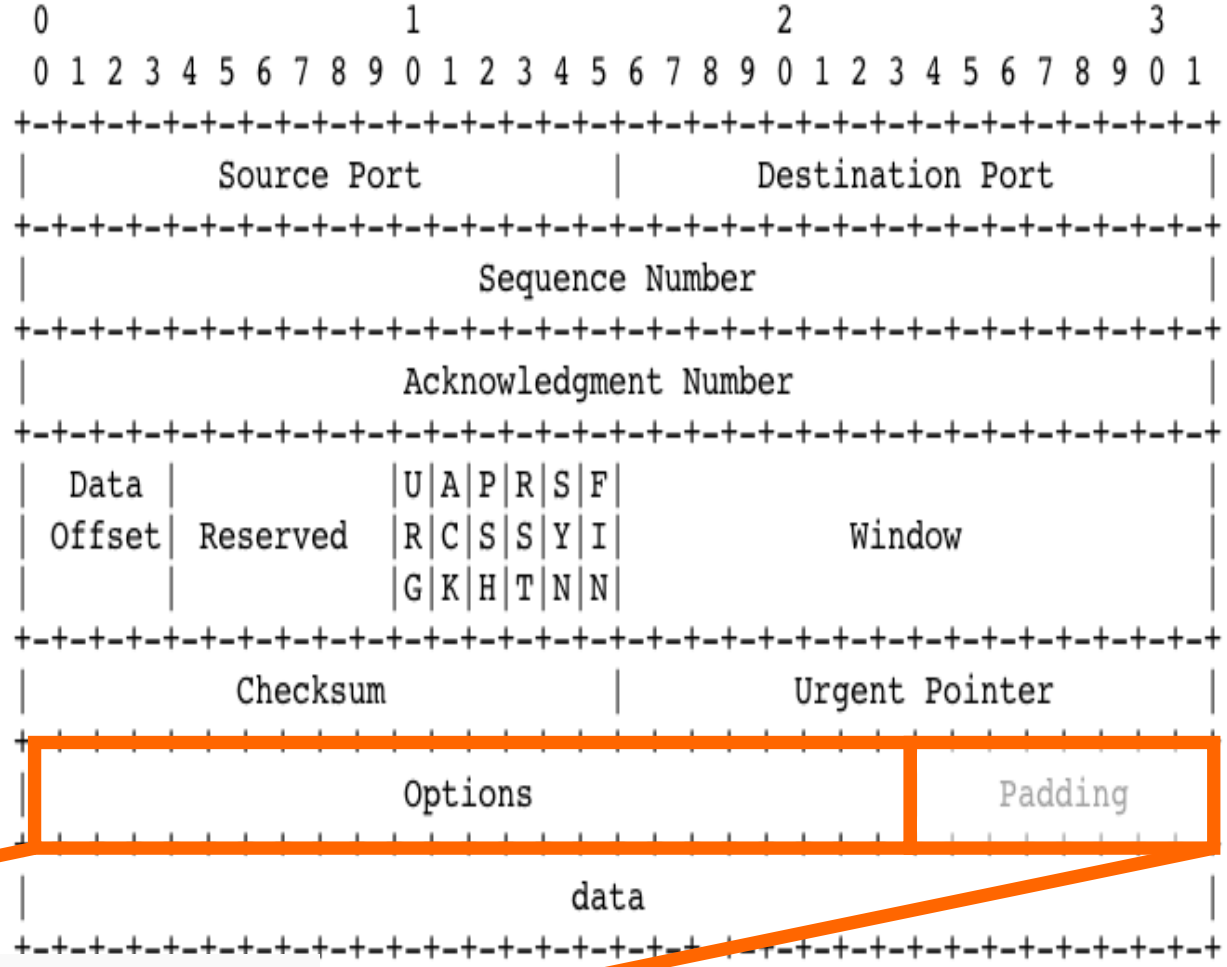
Message Format

Not relevant for the exam

definition of max. segment size

- Kind: 1 byte; Kind == 2 (defined to be “2”).
  - If this option is present, then it communicates the maximum receive segment size at the TCP endpoint that sends this segment.
  - This value is limited by the IP reassembly limit.
  - This field may be sent in the initial connection request (i.e., in segments with the SYN control bit set) and MUST NOT be sent in other segments (MUST-65). If this option is not used, any segment size is allowed.
- Length: 1 byte; Length == 4.
  - Length of the option in bytes.
- Maximum Segment Size (MSS): 2 bytes.
  - The maximum receive segment size at the TCP endpoint that sends this segment





TCP Header Format

Image source: http://www.rfc-editor.org/rfc/rfc793.txt

Not relevant for the exam

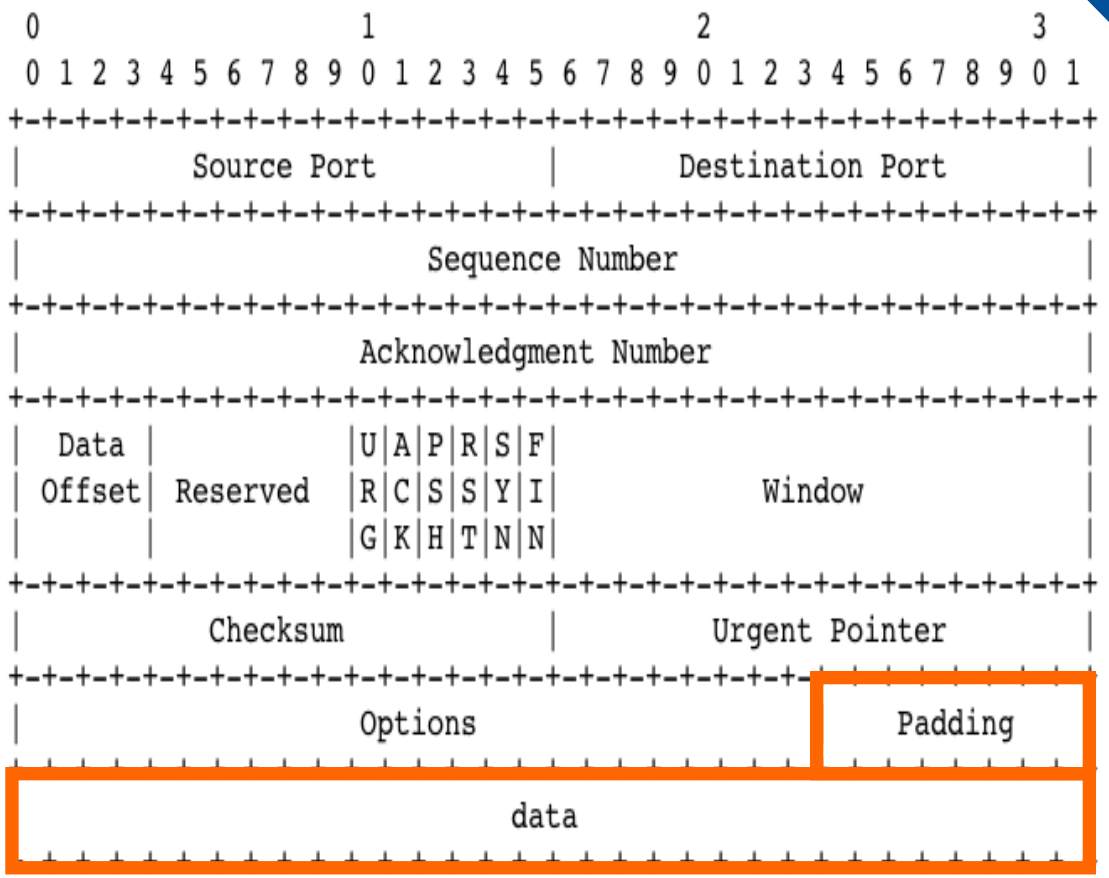
Message Format

Padding

- Zeros to align options to 4 byte boundary

Data

- Application data
  - Application header
  - User data



TCP Header Format

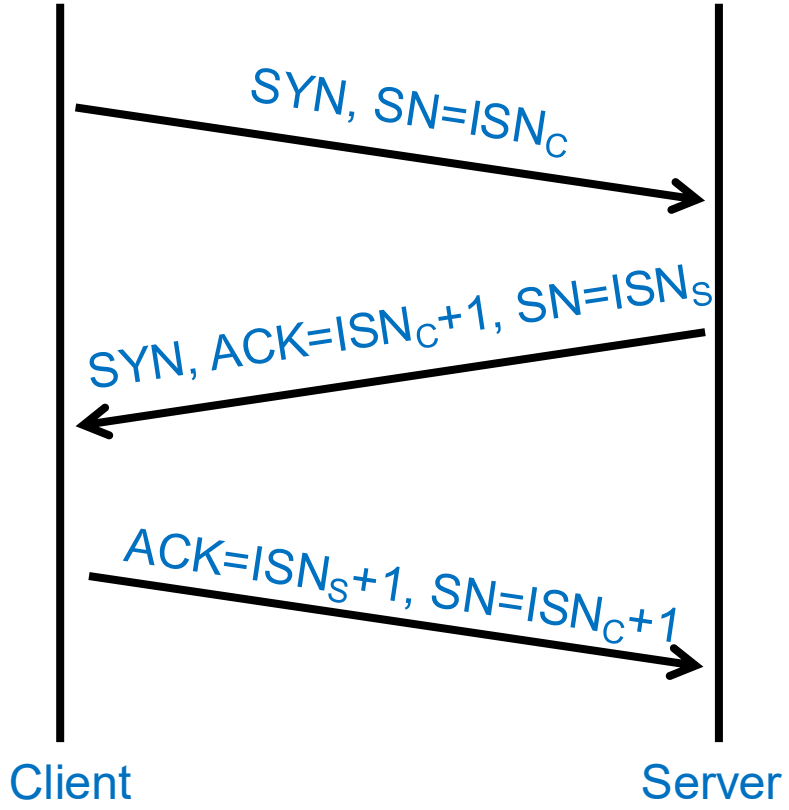
Image source: http://www.rfc-editor.org/rfc/rfc793.txt

Connection Setup

TECHNISCHE UNIVERSITÄT DARMSTADT

TCP connection setup: 3-way handshake

- Step 1:
  client sends message with
  - SYN flag set (as connection request CC)
  - Sequence number (SN) field containing Client Initial Sequence Number (ISN_C)
- Step 2:
  server sends message with
  - SYN and ACK flags set
  - Acknowledgment field containing ISN_C+1
  - Sequence number field containing Server Initial Sequence Number (ISN_S)
- Step 3:
  client send message with
  - ACK flag set
  - Acknowledgment field containing ISN_S+1
  - Sequence number field containing ISN_C+1

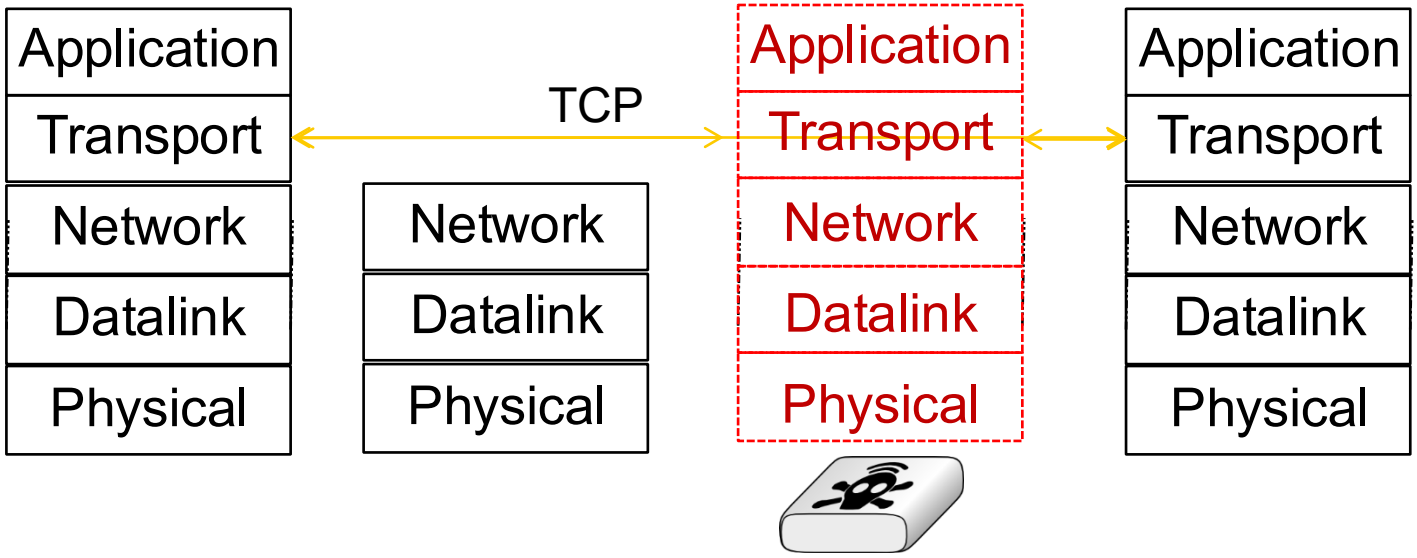


Connection Management

MPTCP is an evolution of TCP

In order that MPTCP behaves like TCP for

- Applications
  - MPTCP supports unmodified applications
- Underlying networks
  - MPTCP works over today's IPv4 and IPv6 networks

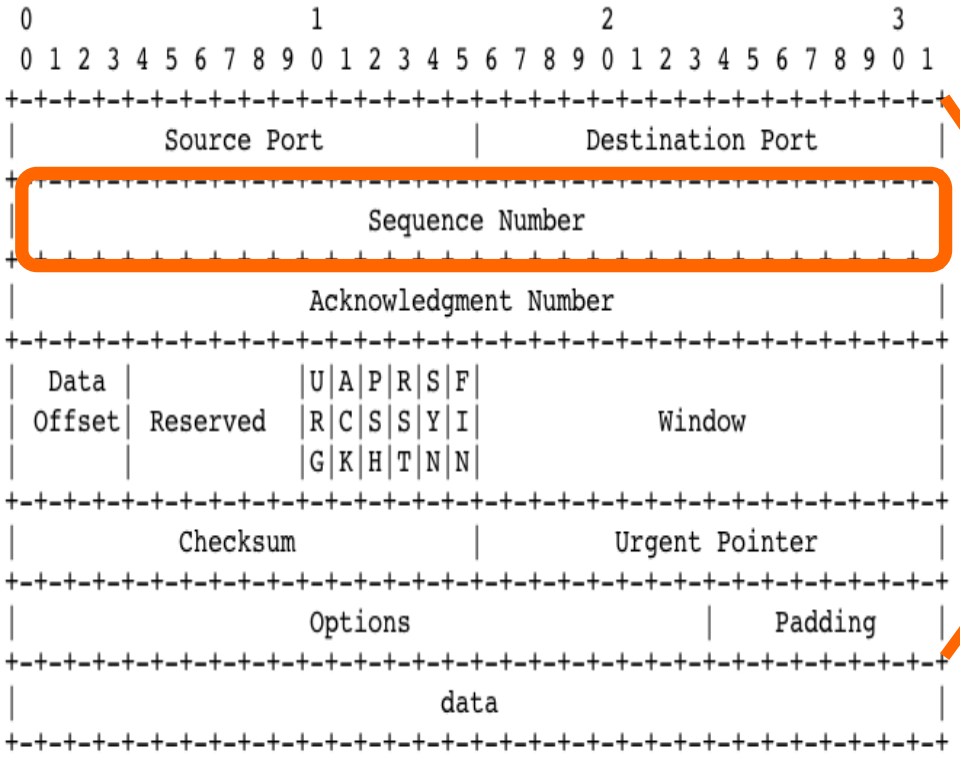


In today's Internet middleboxes exist

- shall see MPTCP data as being a TCP data
- E.g.
  - firewall, NAT, voice gateway, ..

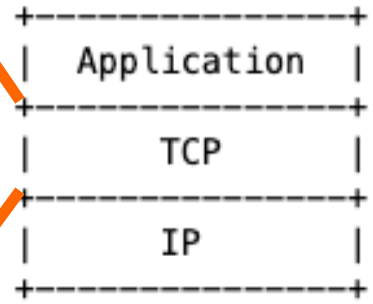
Source: Olivier Bonaventure

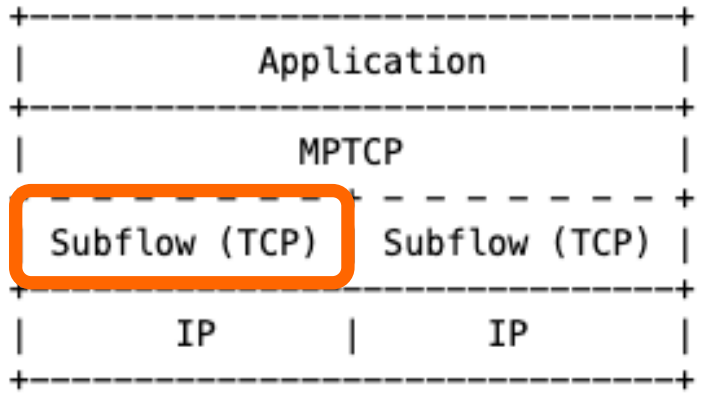
Role of Sequence Numbers and Options



TCP Header Format

Comparison of Standard TCP and MPTCP Protocol Stack





Connection Management

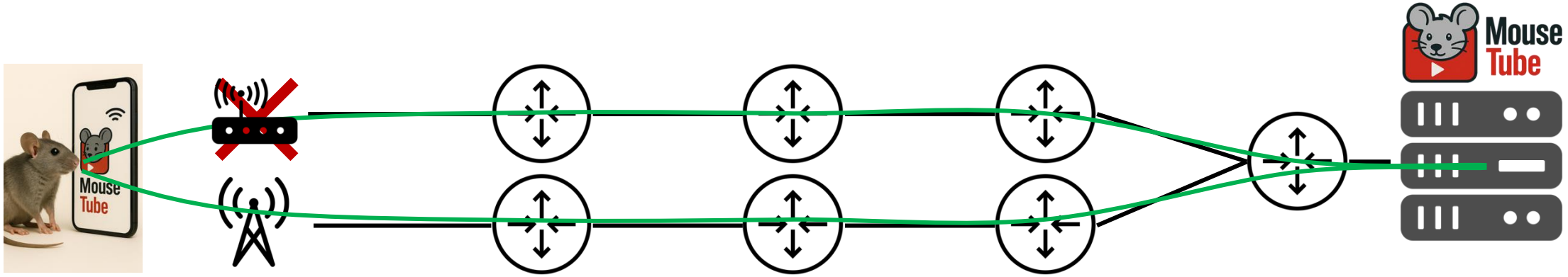
TECHNISCHE UNIVERSITÄT DARMSTADT

Packets use multiple paths and may not arrive in the order they are sent in

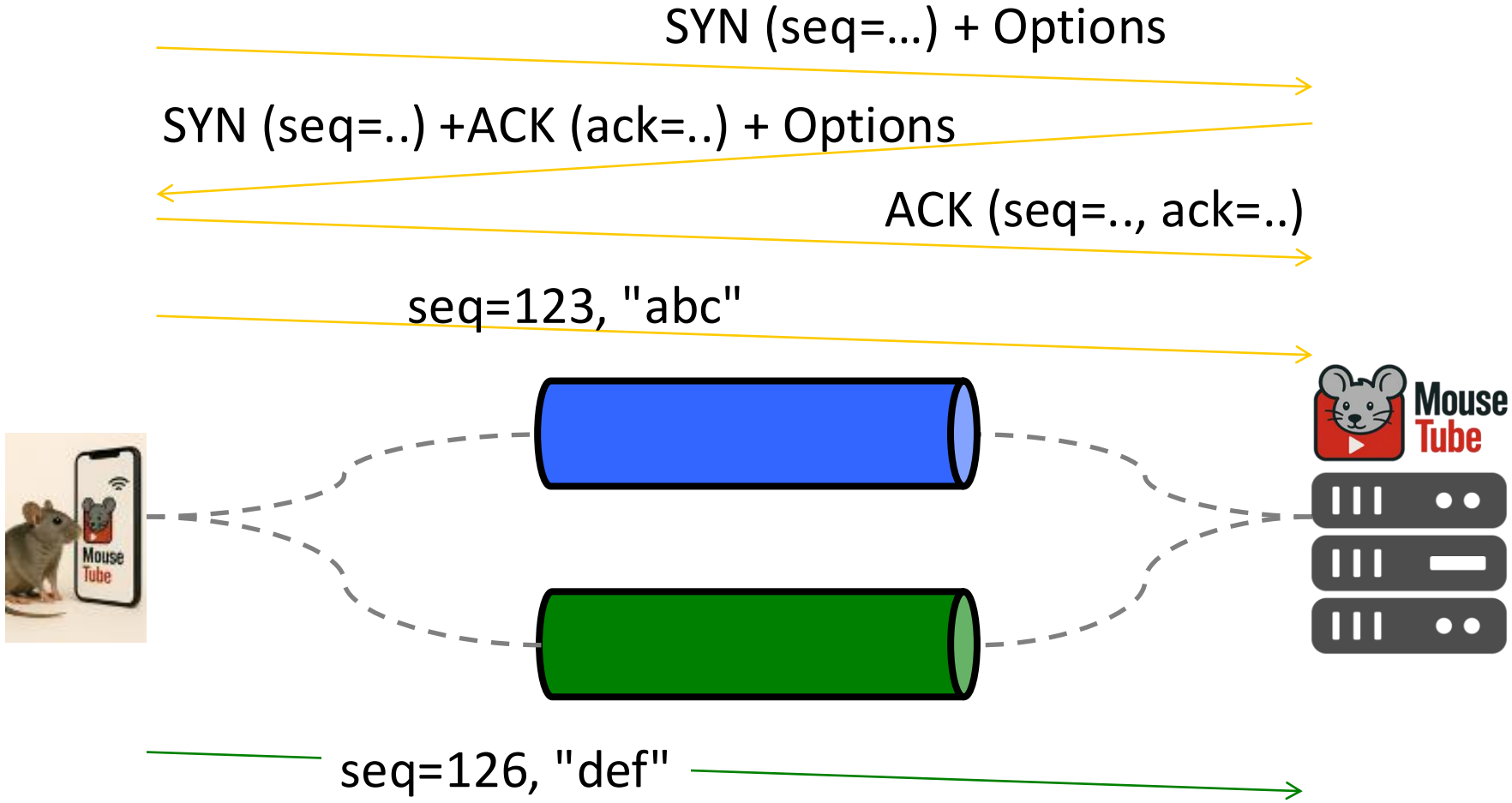
- Round-robin transmission via paths of different latency
- Failure of one path, leading to a retransmission of all packets that were in transit along this path

MPTCP needs additional sequence numbers

- One for the overall MPTCP connection
- One for each individual TCP connection



Connection Management - a naïve Multipath TCP in the “old” Internet



Options specify

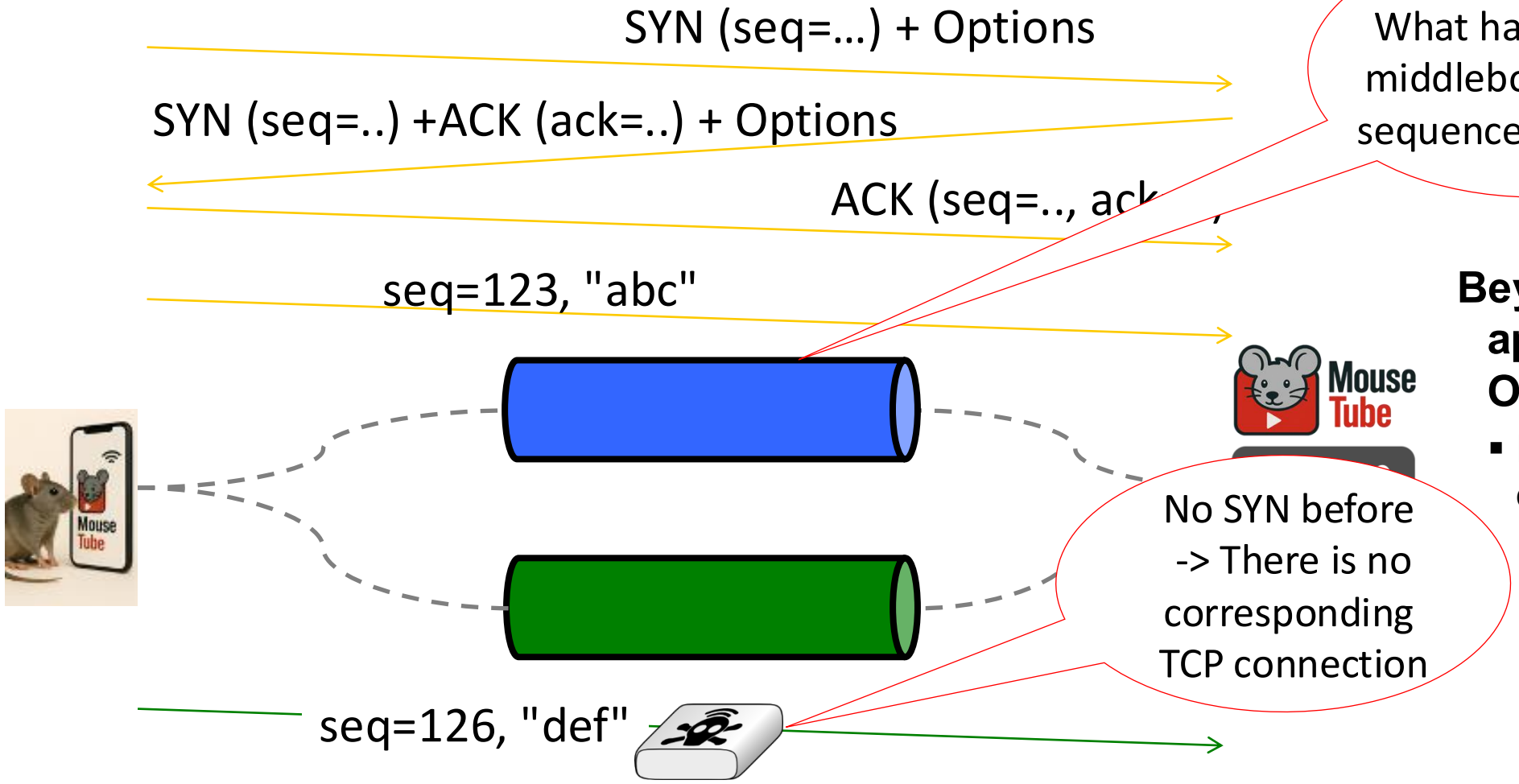
- if MPTCP is available

Beyond the naïve approach: Options used for

- Identification of endsystems

Source: Olivier Bonaventure «Decoupling TCP from IP with Multipath TCP»

Connection Management - a naïve Multipath TCP in the “old” Internet



Beyond the naïve approach: Options used for

- Identification of endsystems

Source: Olivier Bonaventure «Decoupling TCP from IP with Multipath TCP»

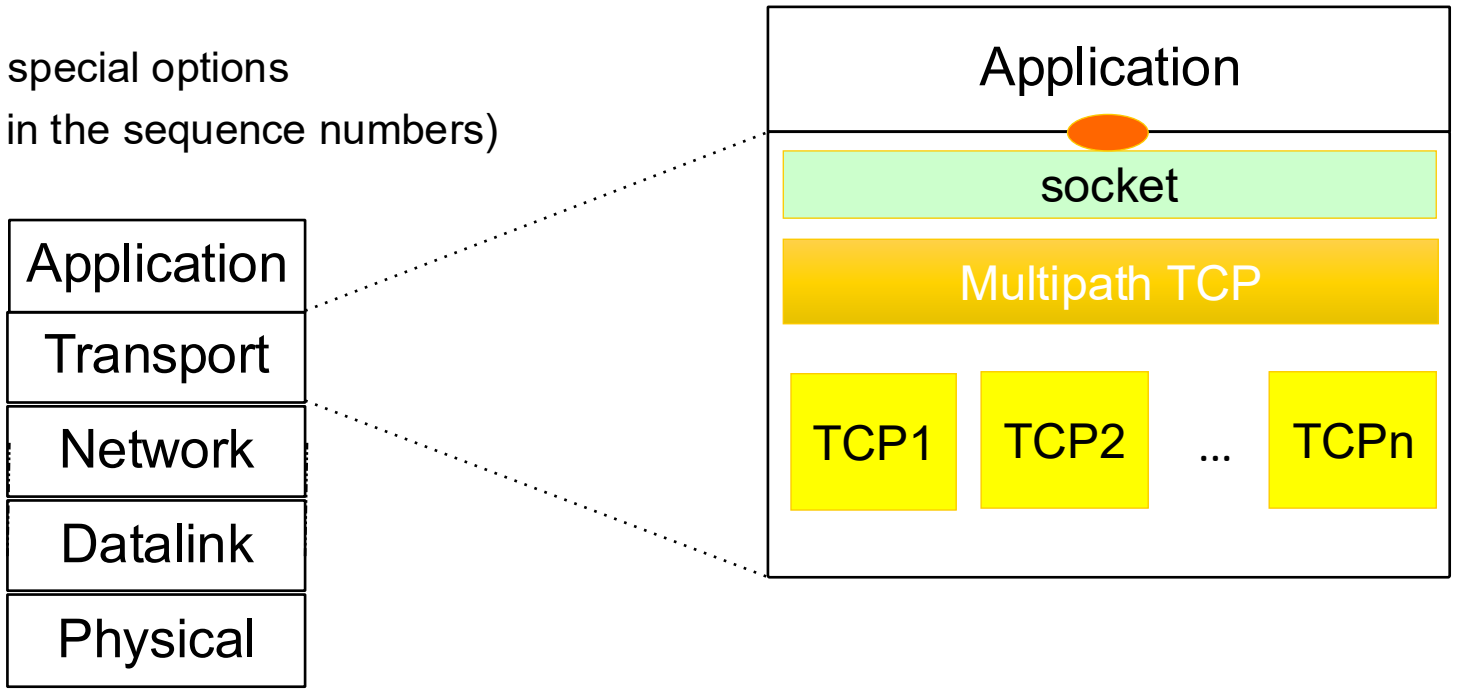
Design Decision

A Multipath TCP connection is composed of regular TCP subflows

- Each TCP subflow
  - is sent over a single path and
  - appears like a regular TCP connection along this path
    - Regular TCP:
      - 3-way handshake, SYN Segments with special options
      - All segments sent in sequence (no gap in the sequence numbers)
      - Terminated using FIN / RST

Challenge:

- How to manage multiple subflows,
  - e.g., ensure in order delivery.

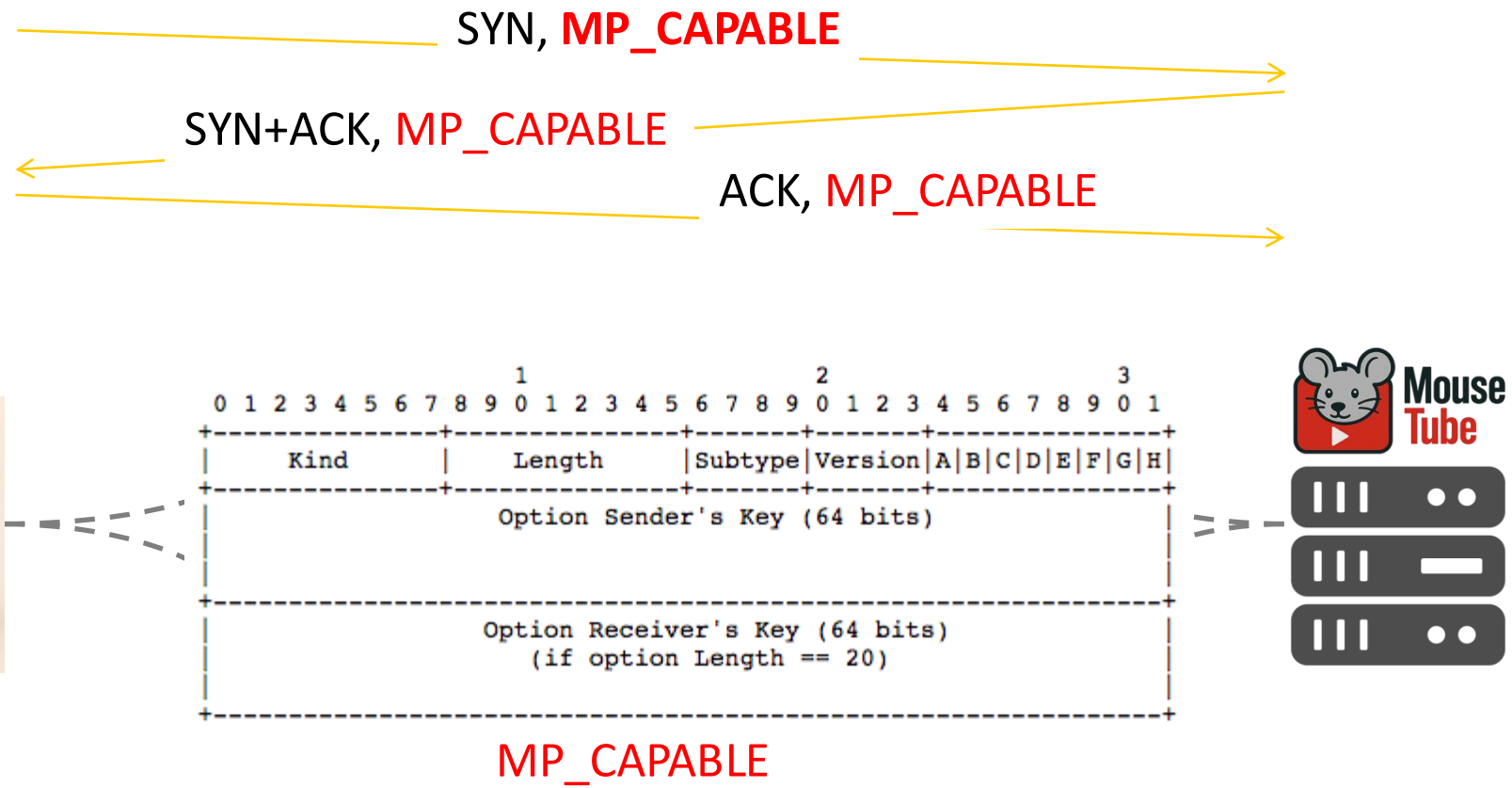


A. Ford, C. Raiciu, M. Handley, S. Barre, and J. Iyengar, "Architectural guidelines for multipath TCP development", RFC6182 2011.

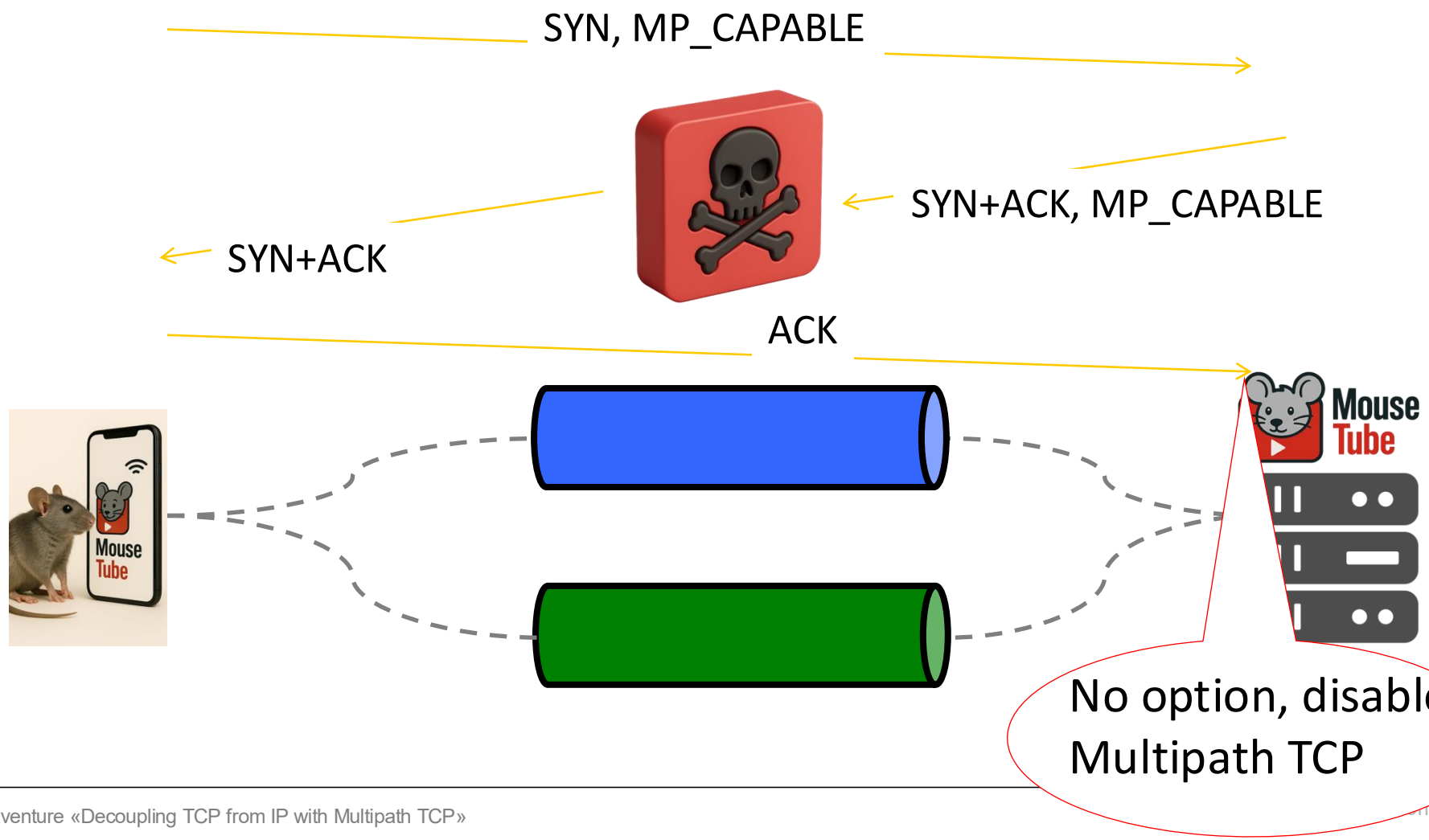
Source: Olivier Bonaventure «Decoupling TCP from IP with Multipath TCP»

Connection Establishment

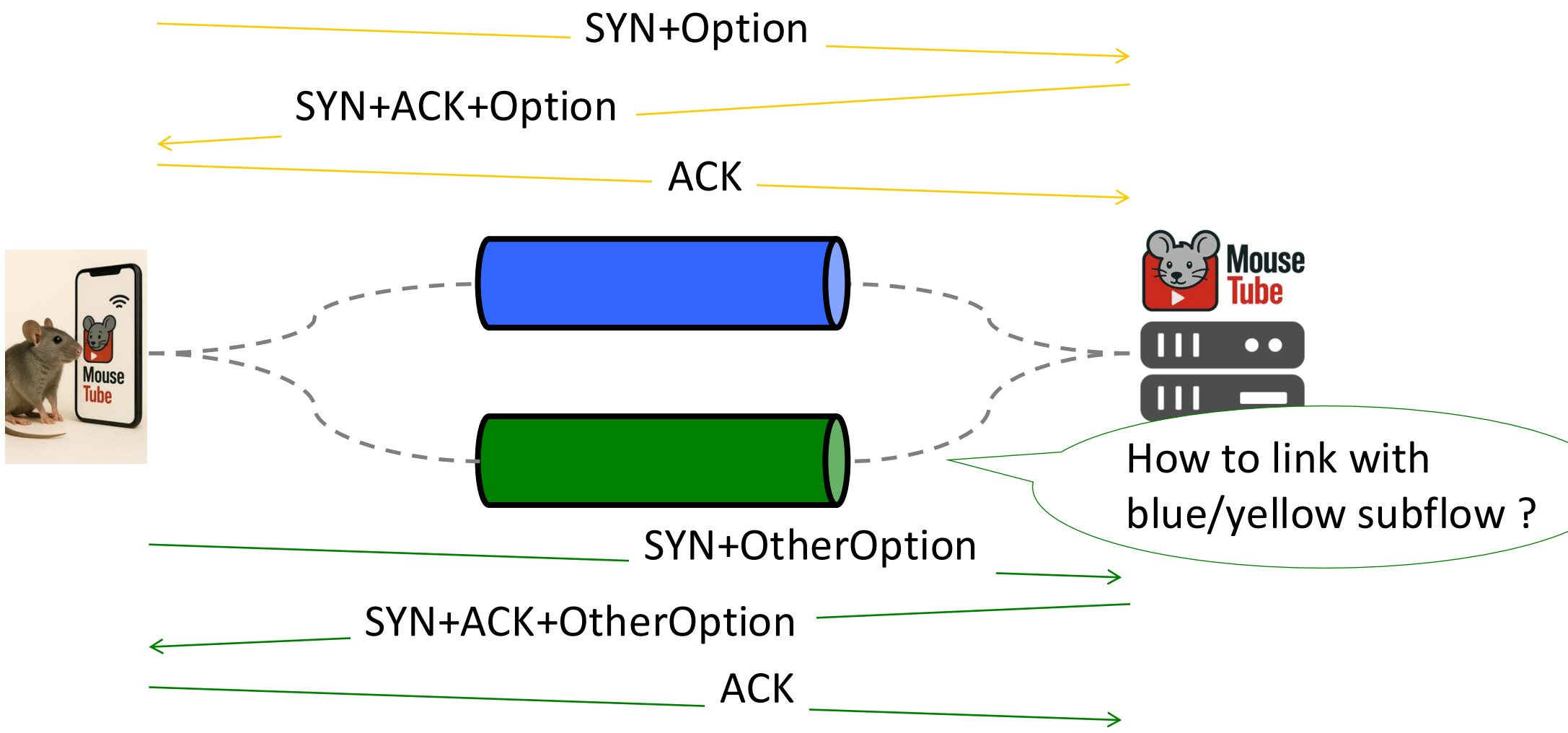
Inform about willingness to open an MPTCP connection: MP_CAPABLE



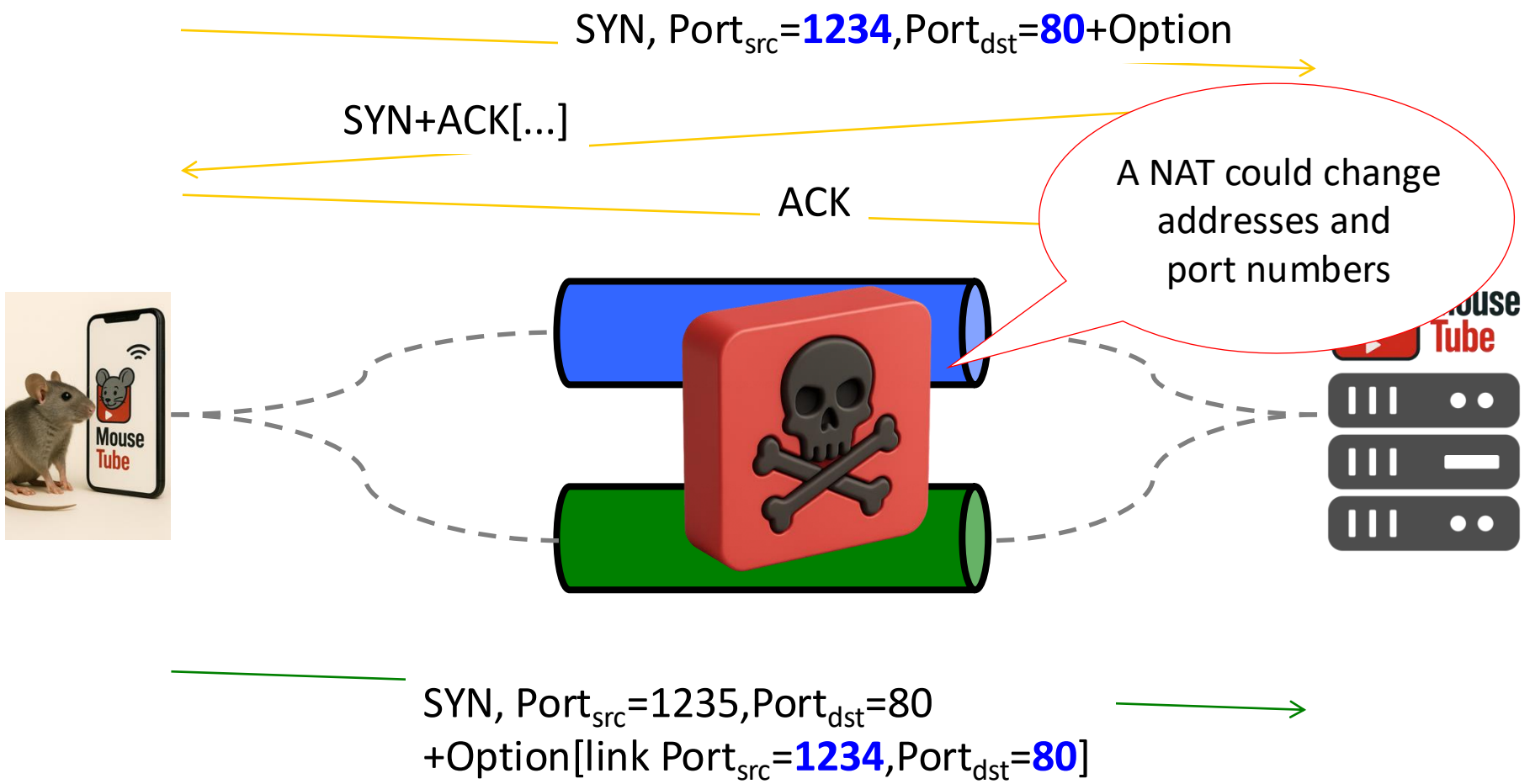
Multipath TCP Option in 3rd ACK



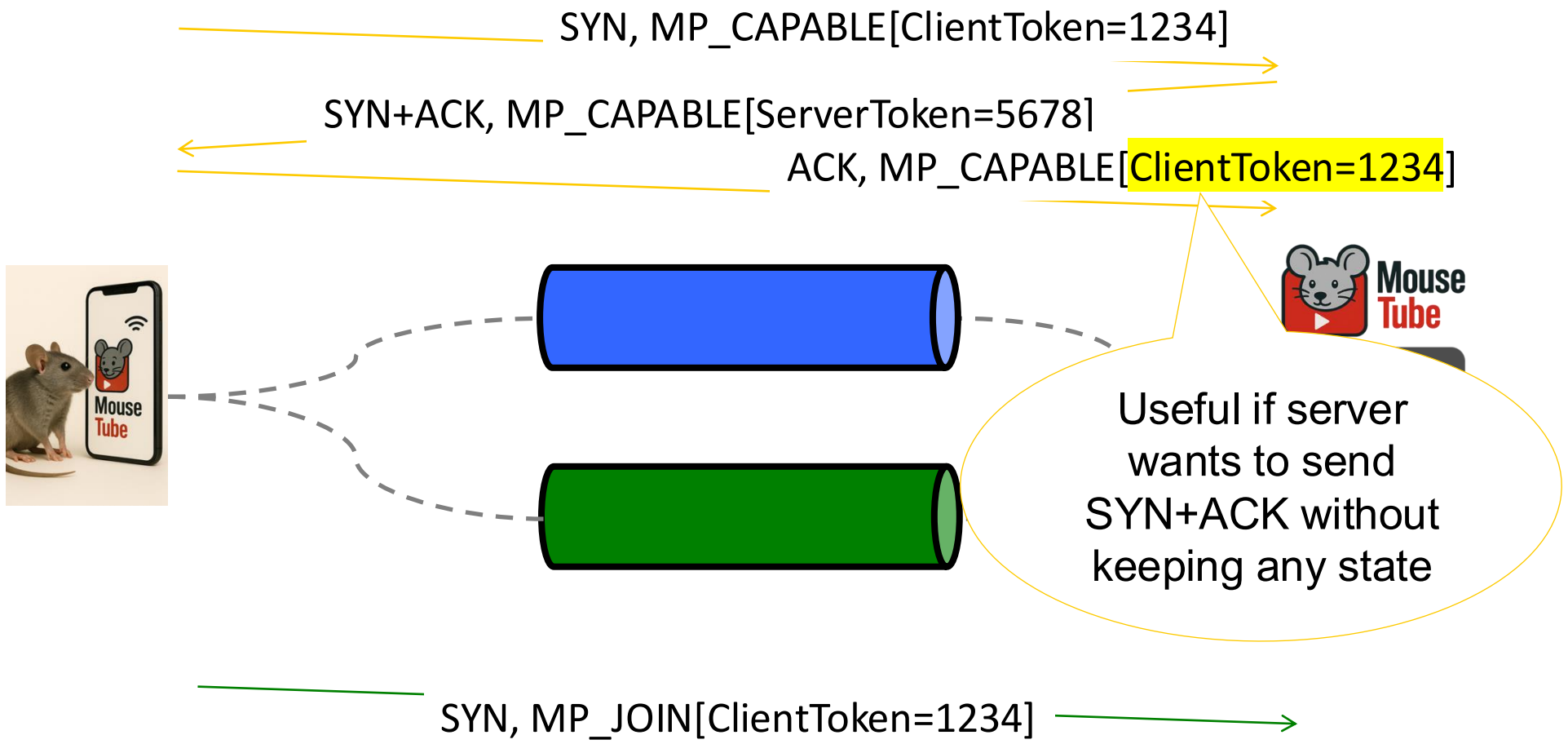
Adding and Removing Subflows



How to Link TCP Subflows?



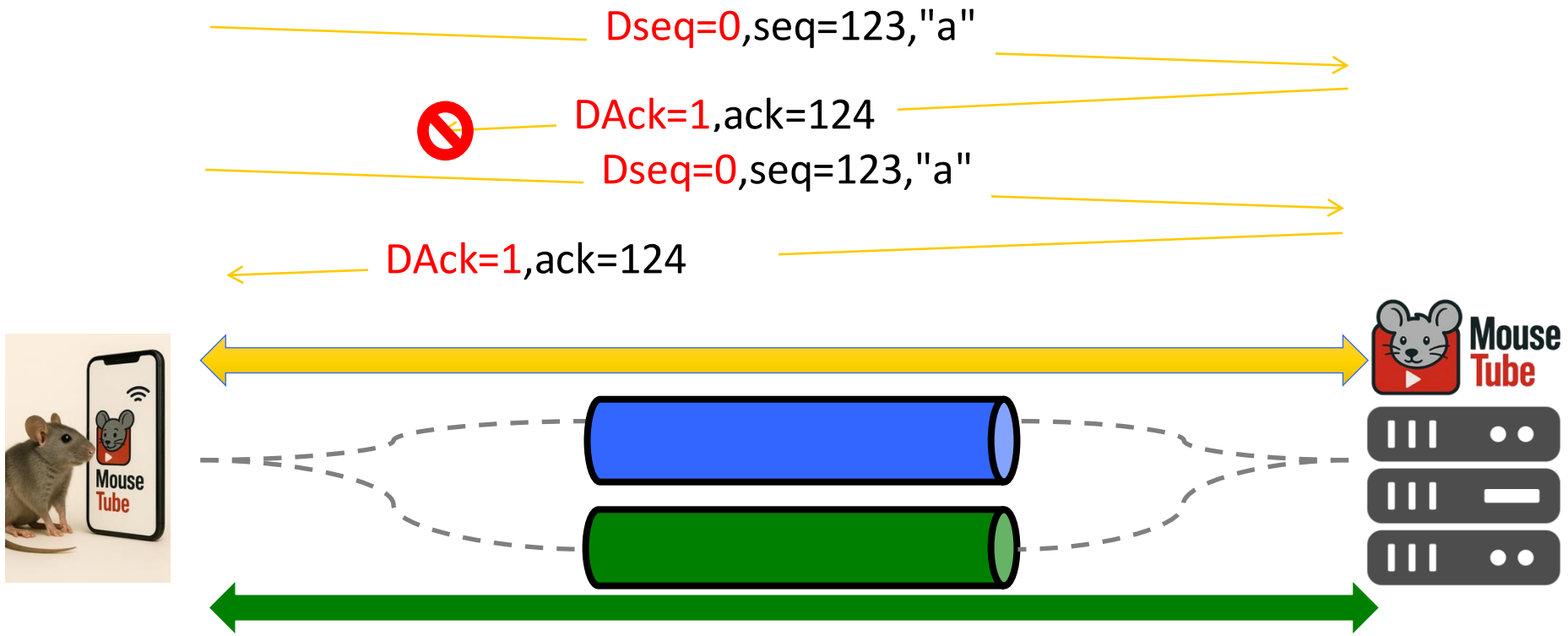
How to Link TCP Subflows?



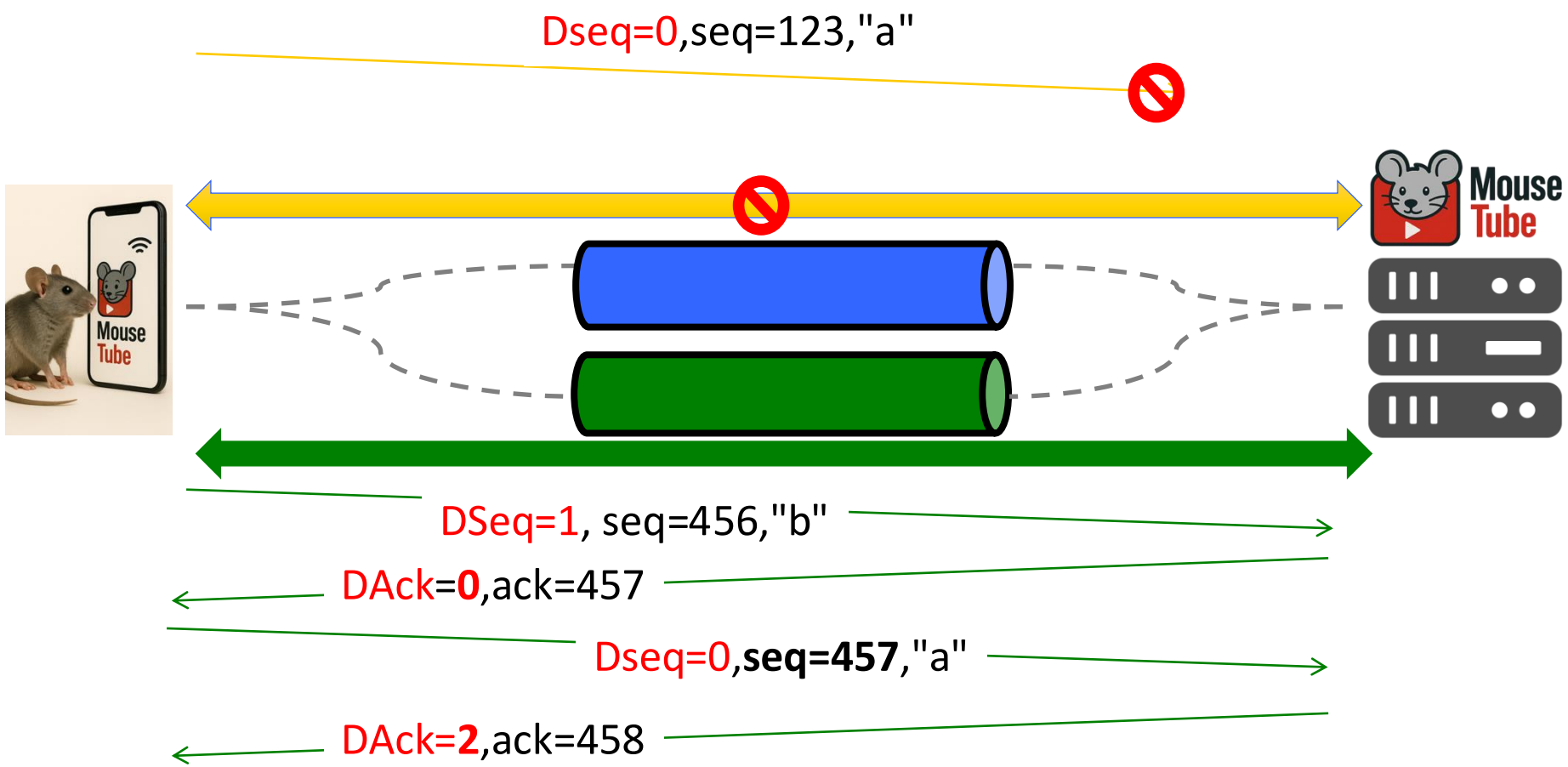
Data Transfer - How to Deal with Losses ?

Data losses over one TCP subflow

- Fast retransmit and timeout as in regular TCP



What Happens when a TCP Subflow Fails?



Retransmission Heuristics

TECHNISCHE UNIVERSITÄT DARMSTADT

Heuristics used by e.g., current Linux implementations

- Fast retransmit is performed
  via the same subflow as the original transmission
- Upon timeout expiration,
  reevaluate whether the segment could be retransmitted over another subflow
- Upon loss of a subflow,
  all the unacknowledged data are retransmitted on other subflows

Source: Olivier Bonaventure «Decoupling TCP from IP with Multipath TCP»

Increasing resilience with MPTCP

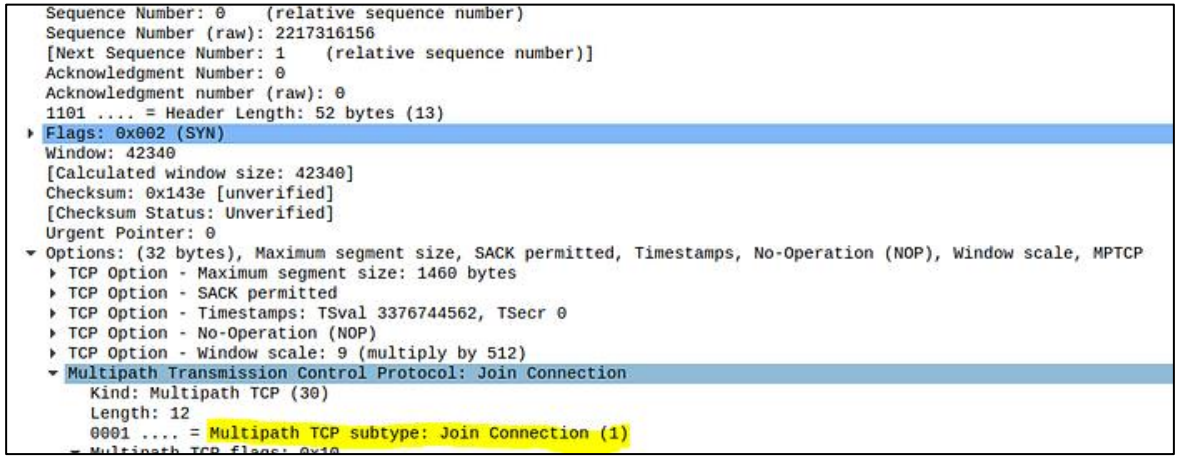
TECHNISCHE UNIVERSITÄT DARMSTADT

MPTCP utilizes several methods to detect failures of a subflow

- Standard TCP methods like Retransmission-Timeouts
- Periodic probes (lightweight heartbeats) to check liveliness

MPTCP can be configured with several transmission strategies

- Use only one path, while the other is kept as backup (similar as in LAG)
- Use all paths simultaneously, but change the scheduling if a flow fails
- Use a redundant scheduler for proactive resilience

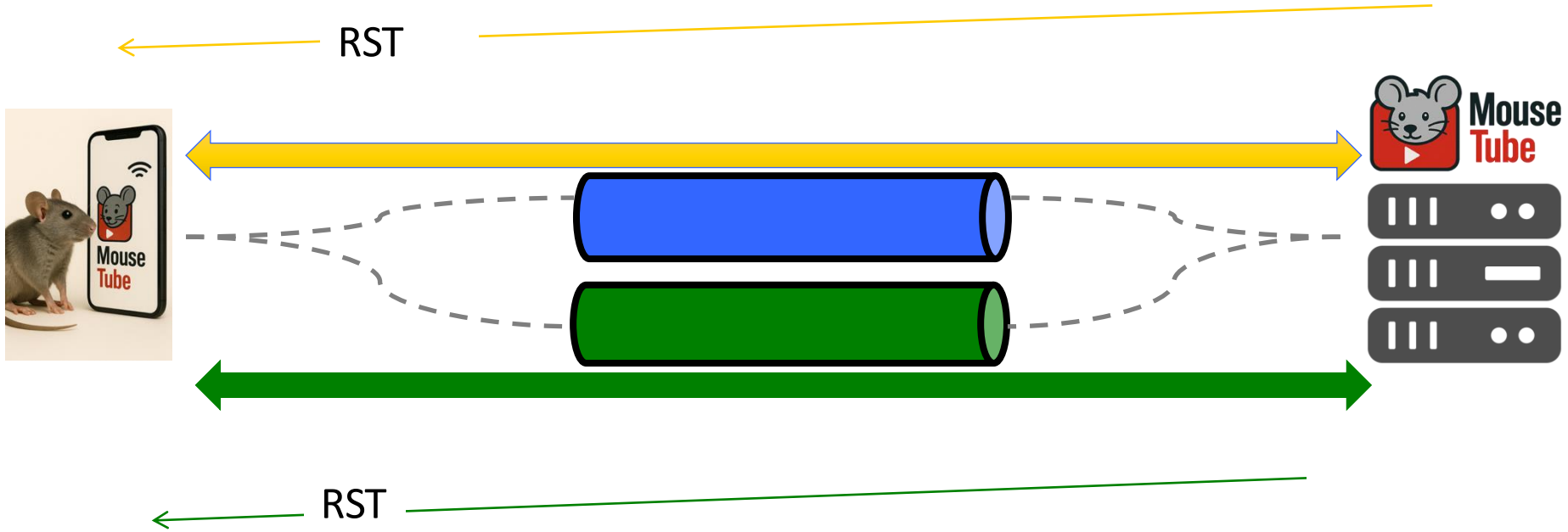


Closing a Multipath TCP Connection

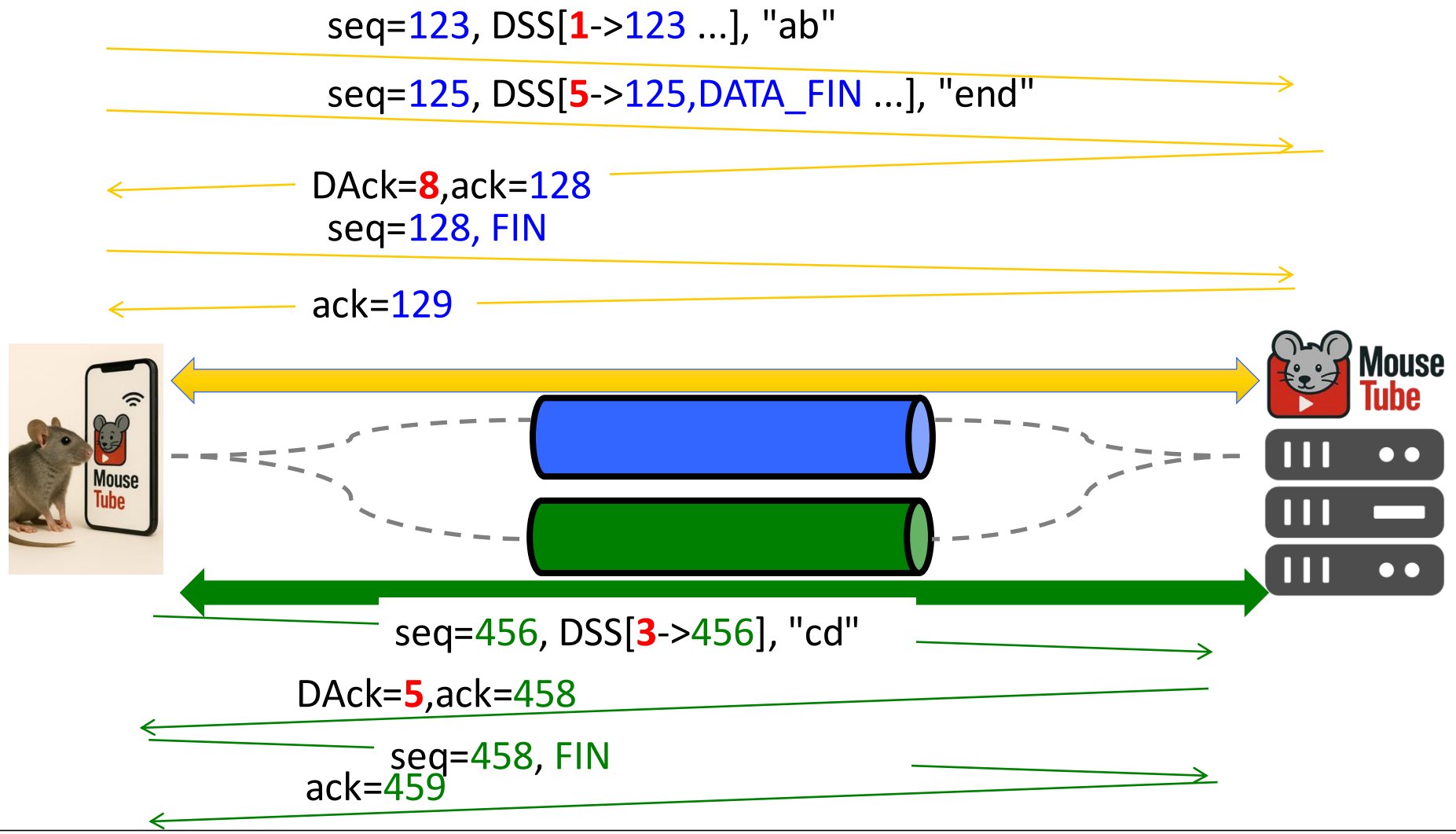
TECHNISCHE UNIVERSITÄT DARMSTADT

How to close a Multipath TCP connection ?

- By closing all subflows ?

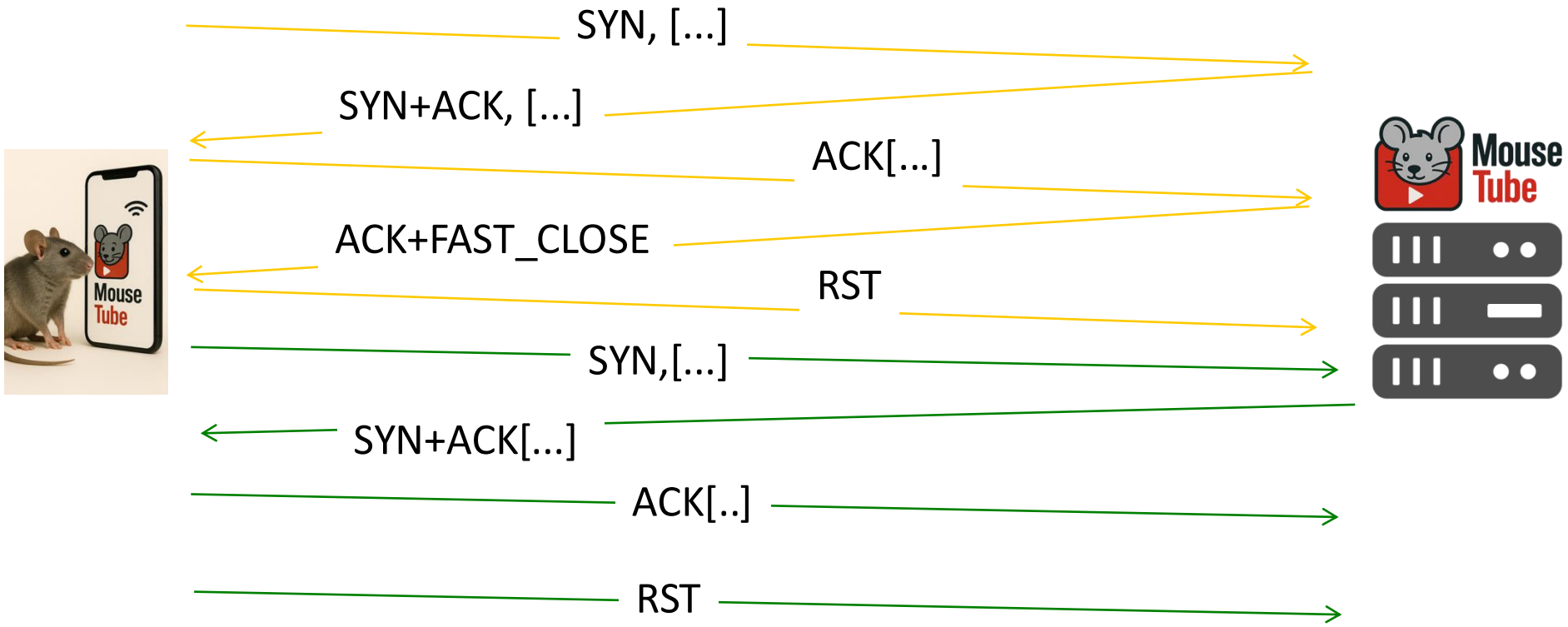


Closing a Multipath TCP Connection



Closing a Multipath TCP Connection

FAST Close

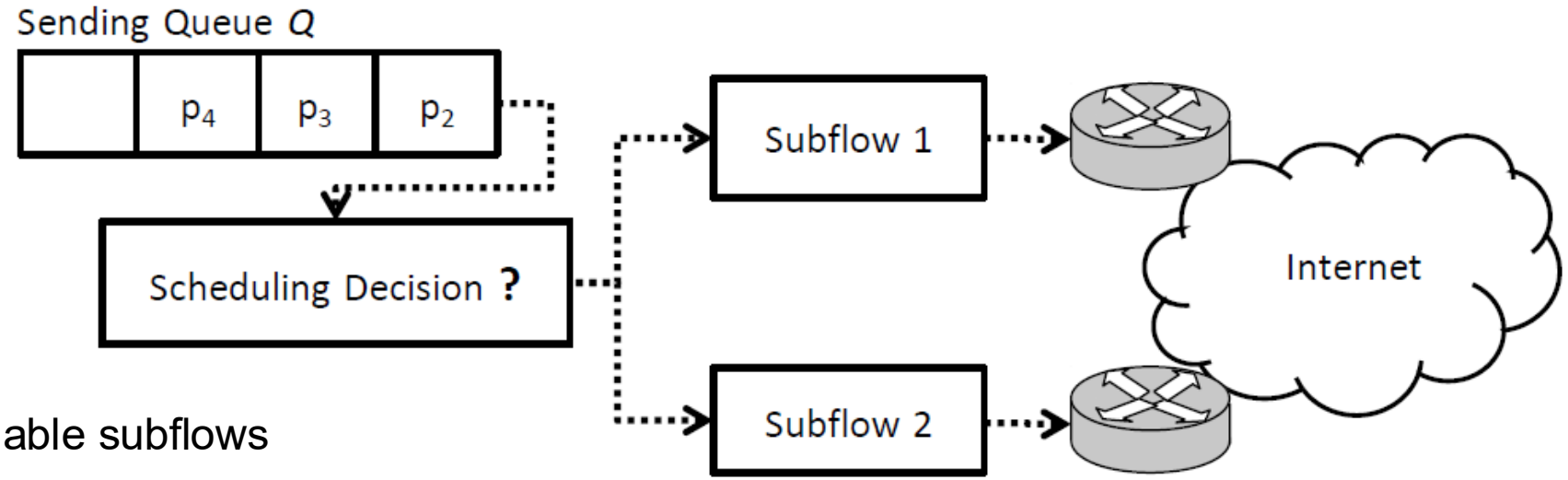


Scheduling

TECHNISCHE UNIVERSITÄT DARMSTADT

How to assign packets to subflows?

- Has huge impact on performance



Simple Idea:

- Round Robin
  - Spread packets on all available subflows

What happens for heterogeneous subflows?

Summary MPTCP

TECHNISCHE UNIVERSITÄT DARMSTADT

MPTCP coordinates multiple TCP connections

- Utilizes unused options in TCP for coordination
- Utilizes data sequence number to coordinate data between TCP connection

Similar to TCP (and in contrast to QUIC), MPTCP is implemented in the kernel

- High level of interoperability, but hard to extend/update
- Easy usability for applications